Supporting information to

YR-MPE: a modular desktop platform for agent-operable and reproducible phylogenetic workflows

YPhylo Workbench and YR-MPE

A User Manual for Molecular Phylogenetics and Evolution Analysis

Version 1.2.0 — November 10, 2026

Zhijie Xu, Yiyang Jia, Zhihan Gu, Mingrui Shang, Xinbo Cheng,
Shuo Han, Yang Jin, Yitong Lei, Rui Zhang, Yunlong Zhang

Department of Biological Sciences and Technology, College of Biological Science and Medical Engineering, Donghua University, Shanghai 201620, China

Department of Cardio-Pulmonary Circulation, Shanghai Pulmonary Hospital, Tongji University School of Medicine, Shanghai 200433, China

Shanghai Center of Systems Biomedicine, Shanghai Jiao Tong University, Shanghai, China

Correspondence: Yunlong Zhang, zhyl@dhu.edu.cn

Contents

1 Introduction **2**

2 Installation and first launch **4**

3 Interface overview **8**

4 Datasets and workspace **11**

5 Alignment **17**

6 Model selection **21**

7 Distance and saturation **23**

8 Phylogenetic inference **25**

9 Species tree and concordance **32**

10 Molecular dating **36**

11 Selection analysis **38**

12 Tree visualization **39**

13 Working with AI assistance **40**

14 Troubleshooting **43**

15 Plugin development **45**

A File formats and conventions **49**

Chapter 1

Introduction

This manual describes YR-MPE, a modular collection of molecular phylogenetics and evolution plugins for the YRTools platform, and YPhylo Workbench, the integrated graphical workbench built on top of those plugins. The first chapters cover installation, the interface and the workspace; the middle chapters are a reference for each analysis menu; the final chapters cover troubleshooting and plugin development. Menu paths are written as **MENU** → **Submenu** → **Action**, and names in `monospace` are files, options or object types. This edition describes YR-MPE 1.2.0. The integrated workbench is called **YPhylo Workbench**; the plugin package remains **YR-MPE**.

1.1 What YR-MPE and YPhylo Workbench are

YR-MPE is a collection of plugins for YRTools, a plugin-based bioinformatics platform. Each plugin wraps one analysis—an aligner, a trimmer, a model selector, a tree program, a dating method, a selection test—and exposes it through a consistent graphical window. YPhylo Workbench is the integrated environment inside YR-MPE. It adds a persistent workspace, typed data objects, automatic hand-off between tools, and a provenance view of everything that has been done to a dataset.

The distinction matters when reading this manual:

- **YR-MPE** is the plugin package. It can be used from the YRTools sidebar one tool at a time, or embedded in another PyQt application.
- **YPhylo Workbench** is the workbench window. It manages sequences, alignments, datasets, models, distance matrices, trees, tree sets and MCMC chains as objects, and passes them between plugins.
- **YRTools** is the host application that loads the plugins and provides the sidebar, settings and update mechanism.

1.2 Design principles

YR-MPE was built around four practical constraints.

Modularity. A method lives in one plugin. Adding or replacing a method does not require rebuilding the workbench. The plugin package is pure Python plus bundled executables, so a repository update can be released without a compilation step.

Reproducibility. Every run records the command that was executed, the input objects, the output files and the tool version. In a persistent workspace these records survive restarts, so a analysis can be inspected or repeated later.

Windows-first packaging. Many phylogenetic programs are developed and tested on Unix-like systems. YR-MPE bundles verified x86_64 Windows builds, including MPI and BEAGLE builds of MrBayes and an MPI build of PhyloBayes, so that the same interface works on a standard laboratory workstation.

Progressive disclosure. Simple analyses use a short form. Advanced options, partition schemes, MPI settings and calibration dialogs are available but do not clutter the default view.

1.3 Two ways to work

There are two complementary workflows.

Standalone plugin mode. Open a plugin from the YRTools sidebar, load input files, set parameters, run the tool, and save the output manually. This is appropriate for one-off analyses and for users who already have a file-based pipeline.

Workbench mode. Open YPhylo Workbench from the YR-MPE sidebar. Import sequences or create a dataset, select an object, and launch a tool from the toolbar. The selected object is imported automatically, and the completed result is added to the workspace as a new object. A pipeline then accumulates as a left-to-right chain: sequences $\rightarrow$ alignment $\rightarrow$ model $\rightarrow$ tree. This mode removes most manual file handling and is recommended for multi-step analyses.

The two modes use the same underlying plugins. The workbench adds persistent storage, dataset and partition management, and automatic result passing.

1.4 License, citation and support

YR-MPE is distributed under the GNU General Public License, version 3. You may modify and redistribute it under the terms of that license. The software is provided "as is", without warranty of any kind. Users are responsible for checking inputs, settings and results; the developers are not liable for decisions made from the output.

When publishing an analysis, cite the method papers for the tools you used, not only the interface. Each plugin's **Citation** tab lists the relevant references with copy buttons. Cite YR-MPE itself by its repository URL until a software paper is available. The bibliography at the end of this manual gives the citations used in the tool sections.

Bug reports and feature requests should include the YR-MPE version, the operating system, the exact command shown in the Console tab, and the smallest input that reproduces the problem.

Chapter 2

Installation and first launch

2.1 System requirements

The pre-built package targets 64-bit Windows 10 and Windows 11. The bundled executables are x86_64 binaries; 32-bit Windows is not supported. Linux and macOS can run the Python plugins and user-supplied binaries, but the packaged Windows executables and the MPI/BEAGLE configuration are Windows-specific.

A practical minimum is 8 GB of RAM for single-gene analyses and 16 GB or more for partitioned phylogenomics, MrBayes with BEAGLE, or PhyloBayes CAT analyses. Long Bayesian runs also need free disk space for chains, checkpoints and traces. A working directory on a local SSD is preferable to a network share or a synchronised cloud folder during a long run.

For manual installation, Python 3.8 or newer is required, together with PyQt5 (including QtWebEngine), Biopython, NumPy, pandas and matplotlib. The repository does not pin exact dependency versions; use current stable releases unless a bundled runtime is provided.

2.2 Installing the pre-built Windows package

1. Download the YR-MPE release archive and the YRTools release archive from their repositories.
2. Unzip both archives to writable locations. Avoid unpacking into `Program Files` unless you have administrator rights; the plugins write configuration and temporary files next to the installation.
3. Place the whole `YR-MPE` folder into the host's plugin directory:

   `<YRTools_root>/plugins/YR-MPE/`

   The folder must keep its internal structure, including `YR_MPE/`, `plugins/`, `bin/`, `settings.json` and `config.json`.
4. Start YRTools by running `launcher.vbs` in the YRTools root directory. The script launches the host without leaving a console window. If your system blocks VBScript, run `python main.py` from the YRTools root instead.
5. Open YPhylo Workbench from the YR-MPE sidebar.

Warning

Do not rename the `YR-MPE` folder or move individual binaries out of `YR_MPE/bin`. The configuration stores paths relative to the plugin root, and the plugin loader resolves the package by name.

2.3 First launch and the Initializer

On first launch, YR-MPE checks whether `YR_MPE/config.json` exists and whether every configured executable can be found. If the file is missing or a path is invalid, the **YR-MPE Initializer** opens.

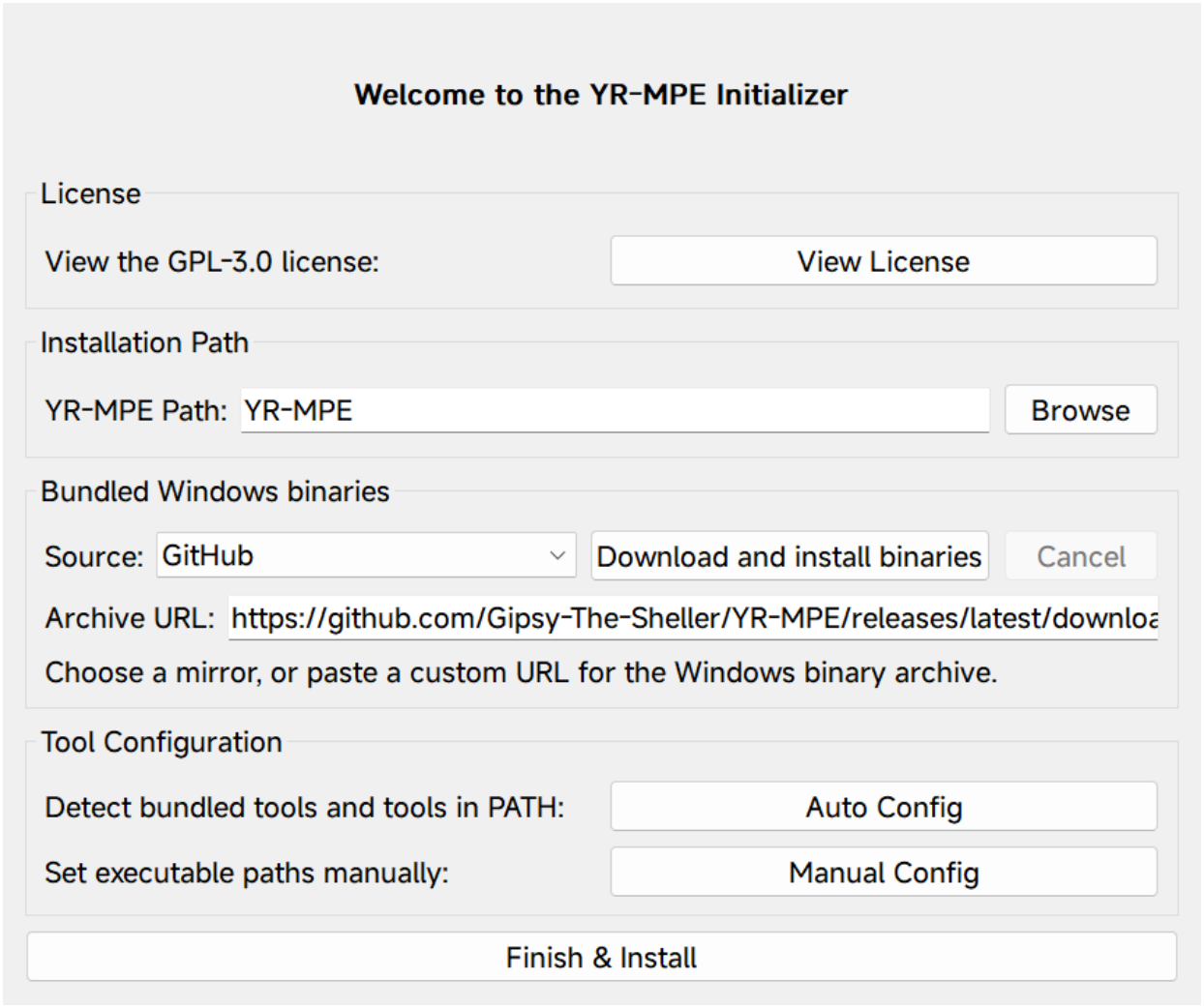


Figure 2.1: The YR-MPE Initializer. The bundled-binary source defaults to GitHub or Gitee; a custom archive URL can be pasted instead. **Auto Config** first checks the extracted `YR_MPE/bin` tree, then the system `PATH`.

The Initializer has five parts:

1. **License.** View the GPL-3.0 text.
2. **Installation path.** The path of the YR-MPE folder relative to the host. The default, `YR-MPE`, is correct for a standard installation under `<YRTools_root>/plugins/`.
3. **Bundled Windows binaries.** Choose the GitHub or Gitee mirror and click **Download and install binaries**. The archive is extracted into `YR_MPE/bin`. If the automatic download fails, paste a direct URL for a `.zip` or `.tar.gz` archive, or download the archive manually and unpack it so that its `bin/` directory replaces `YR_MPE/bin`. The default mirrors are in `binary_sources.json`; edit that file to add an institutional mirror.
4. **Tool configuration. Auto Config** first checks the bundled `YR_MPE/bin` tree, then searches the system `PATH` for each expected command (using `where` on Windows and `which` elsewhere). **Manual Config** opens a table in which you can browse to each executable. Tools that are not found are listed with their download pages.
5. **Finish & Install.** Writes `settings.json` and `YR_MPE/config.json`, then asks you to restart.

A pre-built package usually ships with a working `config.json`, so the Initializer may not appear. If a single tool is later moved or upgraded, edit its entry in `YR_MPE/config.json` rather than re-running the whole wizard. Each entry has the form:

```
{
  "name": "MAFFT",
  "path": "/bin/mafft/mafft.bat"
}
```

The `name` must match the `tool_name` declared by the plugin (case-insensitive), and `path` is relative to `YR_MPE/`.

2.4 Bundled binaries

The Windows package includes the following executables. Versions are those reported by the build shipped with this manual; a plugin's Console tab always shows the actual command and version banner for the current installation.

Program	Typical version	Used by
MAFFT	7.505	ALIGN → MAFFT
MUSCLE 5	5.3	ALIGN → Muscle 5
Clustal Omega	1.2.2	ALIGN → Clustal Omega
MACSE 2	2.07	ALIGN → MACSE
TrimAl	1.2rev57	ALIGN → Trimal
GBlocks	0.91b	ALIGN → GBlocks
IQ-TREE 3	3.0.1	ModelFinder, model parameters, distance, IQ-TREE, concordance factors, RF comparison
RAxML-NG	1.2.2	PHYLOGENY → RAxML-NG
ModelTest-NG	current build	MODEL → ModelTest-NG
MrBayes	3.2.7a (MPI + BEAGLE)	PHYLOGENY → MrBayes3-MPI-BEAGLE; CLOCK → MrBayes
PhyloBayes-MPI	1.9	PHYLOGENY → PhyloBayes-MPI
MPBoot	1.1.1	PHYLOGENY → MPBoot
DecentTree	1.0.0	PHYLOGENY → DecentTree
ASTER	1.24.x	SPECIES TREE → ASTRAL-IV, wASTRAL, ASTRAL-Hybrid, CASTER-site
TreeRecs	1.2	SPECIES TREE → TreeRecs 1.2
LSD2	2.4.4	CLOCK → LSD2

2.5 Manual installation, updating and uninstalling

Manual installation is intended for developers and for users who need to run the plugins inside another PyQt host. Clone YRTools and YR-MPE into `<YRTools_root>/plugins/`, install PyQt5, PyQtWebEngine, Biopython, NumPy, pandas and matplotlib, and run `main.py` from the YRTools root. If the Initializer does not open, copy `config.template.json` to `YR_MPE/config.json` and edit the paths by hand; a plugin can also be launched directly through its `PluginEntry.run()` method. The repository runs from source without compilation, so a new method or bug fix can be distributed by updating the plugin folder.

To update, replace the `YR-MPE` folder with the new release, but keep `YR_MPE/config.json` and `settings.json` if you have customised paths. Workspaces live outside the plugin folder and are unaffected. To uninstall, remove the plugin folder; workspace folders must be deleted separately.

2.6 Verifying the installation

A quick check after installation:

1. Open YPhylo Workbench. The toolbar should show WORKSPACE, ALIGN, MODEL, DISTANCE, PHYLOGENY, SPECIES TREE, CLOCK and SELECTION.
2. Open **ALIGN** → **Open Sequence Files** and load a small FASTA alignment.
3. Run **ALIGN** → **Align by...** → **MAFFT** with the default settings. The Console tab should show the MAFFT command and finish without an error, and the Output Preview should show aligned sequences.
4. Run **MODEL** → **Find Best Substitution Model (ML)** on the same alignment. A ranked model table confirms that IQ-TREE is configured.

If a step fails, the Console tab names the executable and the exact command. Chapter 14 lists the common causes.

Chapter 3

Interface overview

3.1 Main window

The YPhylo Workbench window has four regions:

- a **toolbar** at the top, with one pop-up button per analysis menu and a feature search box on the right;
- a central **workspace grid** of icons, one per data object;
- a right-hand **Details / Selected** panel, opened on demand;
- a **status bar** that reports the current workspace, running tasks and short hints.

The toolbar is the map for the rest of the manual. Hovering a button expands its menu; clicking a menu item opens the corresponding plugin. Disabled items are placeholders for methods that are not implemented in this version and say so in their label or tooltip.

3.2 Toolbar map

Menu	Actions (submenus in parentheses)
WORKSPACE	Select Workspace; Dataset (New, Import from partitioned NEXUS file, Create by SeqMatrix, Create by SeqDBG, One-Step OrthoFinder); Record & Workflow (Import Workflow, Workflow Preview, Run Workflow); About YPhylo Workbench.
ALIGN	Open Sequence Files; Align by... (Clustal Omega, MAFFT, Muscle 5, MACSE); Trim Alignment by... (Trimal, GBlocks).
MODEL	Find Best Substitution Model (ML); Find Best Substitution Model (ModelTest-NG); Estimate Model Parameters (DNA, ML).
DISTANCE	Compute Pairwise Distances (ML); Compute Uncorrected Distances (p-distance); Calculate & Plot Substitution Saturation.
PHYLOGENY	Maximum Likelihood (ML) (IQ-Tree 3, RAxML-NG); Bayesian Inference (BI) (MrBayes3-MPI-BEAGLE, PhyloBayes-MPI, MCMC Diagnostics); Maximum Parsimony (MPBoot); Distance Methods (DecentTree); Tree Viewer (IcyTree); Tree Comparison (Pairwise Robinson–Foulds).
SPECIES TREE	ASTRAL-IV; wASTRAL; CASTER-site; TreeRecs 1.2; Gene Concordance (gCF); Site Concordance (sCF); Site Diagnostics (sCF + CASTER); MSC Diagnostics (gCF + ASTER CU); BEST / MrBayes MSC.
CLOCK	LSD2; MrBayes3-MPI-BEAGLE (dating mode).
SELECTION	Codon-based Z-test (NG86 / YN00); Batch dN/dS (NG86 / YN00 / IQ-TREE); CodeML / HyPhy (planned, disabled).

3.3 Workspace objects and provenance

Every object in the workspace has a type: sequence file, alignment, dataset, model, distance matrix, tree, tree set, chain, table or report. The icon shape and colour identify the type. Objects are arranged in rows by type, with newest objects at the end of the row.

Each object also carries provenance metadata: the run that produced it, the inputs consumed by that run, the tool and version, and the command line. This is what makes the **Details** tree useful. It is a forward view of the provenance DAG: a run is anchored once under its first input, and the remaining inputs are listed in the run tooltip. Reverse edges are not drawn, so every object appears exactly once.

3.4 The common plugin window

Most analysis plugins share the same four-tab window.

Input & Parameters. Supply data and set options. You can use **Browse Files...** (multiple selection is supported for batch runs), drag files into the window, or paste sequences into the **Sequence** text box. For aligners and trimmers, the **Save to** choice controls where the result is written.

Output Preview. Renders the result: a tree in IcyTree, an HTML report, a distance matrix, a model table, a concordance table, or a text preview. When several files were processed in a batch, the **Select Report** drop-down chooses which one is shown.

Console. A colour-coded, timestamped log of the command sent and the program's output. This is the first place to look when something fails. **Clear Console** empties it; **Auto Scroll** toggles following.

Citation. The reference or references to cite, each with a copy button. Citations are resolved from the citation database, not typed into the plugin.

At the bottom of the window are **Run**, **Stop** and a progress bar. When a tool is launched from the workbench, an import button appears after a successful run; its label is **Import to Current Platform** in older panels and **Send to workspace** in newer ones. Both do the same thing: they push the validated result into the workspace as a typed object.

Long-running tools (MrBayes, PhyloBayes, OrthoFinder) open as non-modal windows. You can continue working in the main window while they run; the status bar and the Console tab report progress.

3.5 Input and output conventions

Batch input. A multi-file selection runs the tool once per file. Each run writes its own output, and the Output Preview's **Select Report** drop-down switches between them. Batch mode is intended for per-locus alignments or per-file distance matrices; it is not a way to pool files into one analysis (use a dataset or the Sequence Concatenator for that).

Save to. The two radio buttons have different consequences:

- **Beside input** writes the result next to the input file as `<input_stem><suffix>.<ext>`. For example, `genes.fas` becomes `genes_mafft.fas` with MAFFT.
- **Temporary File** writes to a temporary directory: the workspace `temp/` folder when the plugin was opened from YPhylo Workbench, otherwise the system temporary directory. Pasting sequence text forces temporary-file output.

The suffix is editable in most aligner and trimmer panels. Output is always written by the plugin, not by the external program's own output option, so the suffix only affects the file name, not the format.

Import versus export. Importing sends an object to the workspace and makes it available to downstream tools. Exporting writes a file that is not tracked by the workbench. Use import when the result is part of a pipeline; use export when you need a file for another program or for archiving.

3.6 Selection and result passing

Clicking an icon selects it and marks it green. The green object is the active input for the next tool. When several objects are selected, the host chooses the most relevant one for the tool being opened; for example, a green model plus a green alignment applies the model to that alignment in IQ-TREE. Tooltips describe the multi-input logic.

Data icons that belong to the same provenance chain (same dataset and `ownership_uuid`) form a visual chain. Hovering one icon grey-highlights every icon on that chain; moving away restores the normal selection colours. Clicking an icon clears the previous chain highlight, keeps the clicked icon green, highlights the rest of the chain in blue, and slides the right-hand inspector open (220 ms, cubic ease-out). The green selection state is preserved for plugin import. Double-clicking opens the object's viewer; the right-click **Deselect** action clears both the chain highlight and the selection. The **Details** toolbar button uses the same slide animation when it is toggled.

The right pane is a vertical splitter with **Details** (the provenance inspector) above **Selected**. Both headers stay visible; only one body is expanded at a time, and the other collapses to its header. **Selected** is a multi-object basket: right-click an icon and choose **Add to Selected**, or drag the icon onto the **Selected** toolbar button. Basket objects stay green across chains and types; selections are written to the workspace state. The panel lists the basket as a type-grouped tree (alignments, trees, tree sets, models, distances and chains, with counts), and the **Send to** tree is grouped by toolbar menu. A single click sends the basket to a tool. Recommendations follow the types present: a tree set offers ASTRAL-IV and wASTRAL, a tree set plus a tree offers gCF/sCF, alignments offer IQ-TREE or CASTER-site, and two or more trees offer the Robinson–Foulds comparison.

3.7 Console conventions

The Console is a record, not a terminal. Command lines are shown in a distinct colour, normal output in the default colour, warnings in amber and errors in red. The timestamp is the time at which the line was received, not the time at which the external program wrote it.

The first line is usually the exact command. If a run fails, compare that command with the tool's own documentation before changing GUI settings. A non-zero exit code is reported with the last lines of stdout and stderr; a tool that writes a valid report and then exits non-zero (ModelTest-NG is one example) is accepted only when the report file exists.

Chapter 4

Datasets and workspace

4.1 Workspace modes

By default no workspace folder is selected, and YPhylo Workbench runs in **Temporary mode**. Objects are kept in memory and in a session temporary directory, and everything is discarded when the window closes. Temporary mode is convenient for a quick look at a file, but it is not suitable for long analyses or for work you may want to revisit.

WORKSPACE → Select Workspace → Select Folder... switches to a **persistent workspace**. The folder becomes a project with three subdirectories:

- `temp/` — the working directory passed to external programs;
- `history/` — a log of every output file and run;
- `results/` — final results and exported objects.

The workspace state is stored in `workspace.json`. Recent workspaces reappear in the sub-menu, and the current one is suffixed **Current**. Switching back to **Temporary** clears the current workspace data after a confirmation prompt.

MrBayes and PhyloBayes require a persistent workspace. Their chains are long, often hours to days, and are checkpointed to disk so that an interrupted run can be resumed. The plugin refuses to start, or warns and asks you to select a folder, when the workspace is temporary. The same applies to any tool whose output you want to keep across sessions.

Use one persistent workspace per project on a local disk: a workspace synchronised by OneDrive, Dropbox or a network share can produce file-lock and permission errors during long Bayesian runs.

4.2 Datasets and partitions

A **dataset** is a set of per-locus partitions treated as one analysis unit. It is the object you send to a partition-aware tool. Create one in any of these ways:

- **New** creates an empty dataset; use the dataset manager's + button to add FASTA files, and **View** to inspect a locus in the sequence viewer.
- **Import from partitioned NEXUS file** parses a NEXUS supermatrix and its `charset` definitions into a dataset.
- **Create by SeqMatrix** builds a dataset from a taxon × locus matrix, including sequences downloaded from NCBI.
- **Create by SeqDBG** builds a dataset from annotated organelle genomes after gene-name standardisation.
- **One-Step OrthoFinder** builds a dataset of single-copy orthogroups from GenBank or proteome input.

The dataset manager controls two independent partition properties:

Topology. *Topo linked* forces one shared tree topology across partitions; *Topo unlinked* lets each partition have its own tree. Concatenated analysis assumes linked topology. The default is unlinked to prevent unintentional concatenation of loci with discordant histories.

Branch lengths. *Edge linked* assumes branch lengths are shared or proportional across partitions; *Edge unlinked* allows independent branch lengths. Edge linked is a stronger hypothesis and is appropriate for partitions with similar rates; edge unlinked is appropriate for heterogeneous partitions.

When a dataset is imported into ModelFinder, IQ-TREE, MrBayes or another partition-aware tool, the supermatrix and partition scheme are built automatically. The taxon set of the supermatrix is the **union** across partitions; a taxon missing from a locus is padded with `?` for that locus, not dropped. Do not assume that the first partition's taxon list is the full taxon set.

4.3 Chain feedback and the inspector

The workflow tree in **Details** is a forward view of the provenance DAG: a run is anchored once under its first input, the remaining inputs are listed in its tooltip, and reverse edges are omitted so every item appears once. Clicking a run node shows the tool, version, command and output files. Right-clicking an object offers **Show**, **Preview**, **Add to Selected**, **Deselect** and, for dating inputs, **Import to LSD2** or **Import to MCMCTree**; the latter is a placeholder in this version and reports that the import will be wired through PD-Guide.

4.4 Sequence viewer

ALIGN → Open Sequence Files opens the sequence viewer, a spreadsheet-style window for one or more sequences.

It opens FASTA or alignment files, inserts sequences from another file, edits residues or whole rows in a grid, switches nucleotide/protein colouring and font, navigates by sequence and column, launches MAFFT, Clustal Omega, MUSCLE 5, TrimAl or GBlocks from its toolbar, and saves or exports the alignment.

Editing is in memory until **Save** or **Save As**; closing without saving loses the edits. Inserting sequences can change the alignment width, so check that every row stays rectangular. The viewer is display-oriented: its colour schemes do not affect exported sequences, and launching a plugin from the viewer creates a new result object rather than silently overwriting the viewer's data.

4.5 SeqMatrix

WORKSPACE → Dataset → Create by SeqMatrix opens a drag-and-drop matrix builder for assembling partitioned multi-locus datasets. Rows are taxa, columns are gene partitions, and cells hold the sequence of one taxon for one locus. SeqMatrix is for assembly, concatenation and partition-definition generation; sequence statistics are computed by downstream tools.

To build a dataset:

1. Add sequences with **Import from Sequences** (multi-FASTA), **Import from NEXUS**, **Import from Excel** (column 1 = name, column 2 = sequence), or **Download from NCBI** for selected accession cells. Imported sequences appear in the left-hand **Sequence Explorer** tree.

2. Drag sequences from the explorer into the cells of the matrix. Use **Append New Row** and **Append New Column** to extend the table; new columns are headed `Partition 1`, `Partition 2`, and so on. The first column, **Sequence Name**, holds the taxon names.

3. Double-click a column header to rename a partition and paste its accession list. Double-click the **Sequence Name** header to edit taxon names in bulk.

4. When the matrix is complete, click **Import as a dataset** to push the concatenated supermatrix and its partition definitions into the workspace, or **Export** to write one FASTA per partition to a folder. A partitioned NEXUS file is generated for downstream partition-aware tools.

Cell colour codes:

Colour	Meaning
Light green	Sequence used once; valid.
Light blue	Sequence downloaded from NCBI.
Light red	Sequence duplicated across multiple cells. This is a warning: an accidental locus duplication biases concatenation and downstream phylogenetic inference.

SeqMatrix only concatenates: sequences in one column must already be aligned to the same length, and names must be unique within a partition. If a cell is filled by accession, the NCBI download must succeed before import.

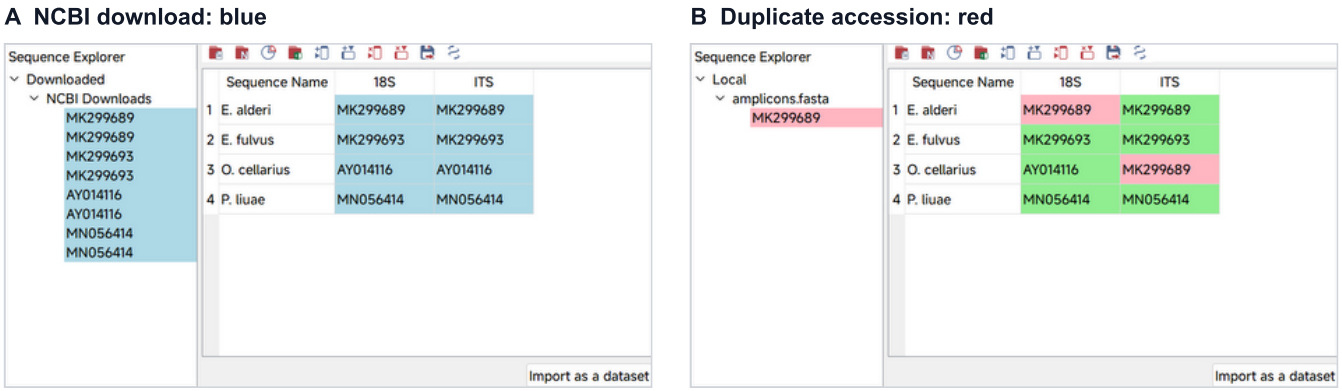


Figure 4.1: SeqMatrix colour states. (A) Blue cells mark sequences retrieved from NCBI by accession. (B) Red cells mark the same accession placed in two cells, which would duplicate a locus in the concatenation. Green cells are ordinary single-use sequences.

4.6 SeqDBG

WORKSPACE → Dataset → Create by SeqDBG opens SeqDBG, a sequence-database visualisation tool for annotated genomes. It reads several GenBank files, builds an adjacency graph of neighbouring genes, and helps identify structural variation and annotation inconsistencies across (typically mitochondrial or chloroplast) genomes. It can then export a standardised per-locus dataset.

SeqDBG is an interactive window rather than the standard four-tab plugin.

1. Click **+ (Add GenBank Files)** and select one or more `*.gb`, `*.gbk` or `*.gbff` files. They are parsed in the background; each taxon appears in the **Taxon Management** list. Annotation classes are colour-coded in the annotation table: CDS green, tRNA purple, rRNA orange.

2. The **Gene Statistics** table lists every gene with its left and right neighbours (**LN**, **RN**) and left, right and taxon counts (**LT**, **RT**, **Taxon**). The counts are heat-coloured so that conserved adjacencies and rearrangements stand out. A low **Taxon** count marks a rarely annotated or non-standard name; low **LT**/**RT** marks a terminal gene or an inconsistent annotation; low **LN**/**RN** marks a structurally conservative adjacency.

3. Click a gene row to draw a three-layer Sankey plot of its left → current → right neighbours on the **Plot** tab. The widths show how often each adjacency occurs.

4. On the **Standardization** tab, review or extend the name-standardisation rules. Built-in defaults map synonyms such as `cox1` → `COX1` and `nad1` → `NAD1`. Rules can be imported and exported as a tab-separated file. Normalisation changes labels, not the underlying sequences.

5. On the **Dataset** tab choose what to export: **Nucleotide** (CDS, tRNA, rRNA, Others) or **Amino Acid** (CDS); choose **All loci** or **Customize loci**; then **Export (FASTA)** to a folder or **Export to YPhylo Workbench** to send the dataset straight to the workspace.

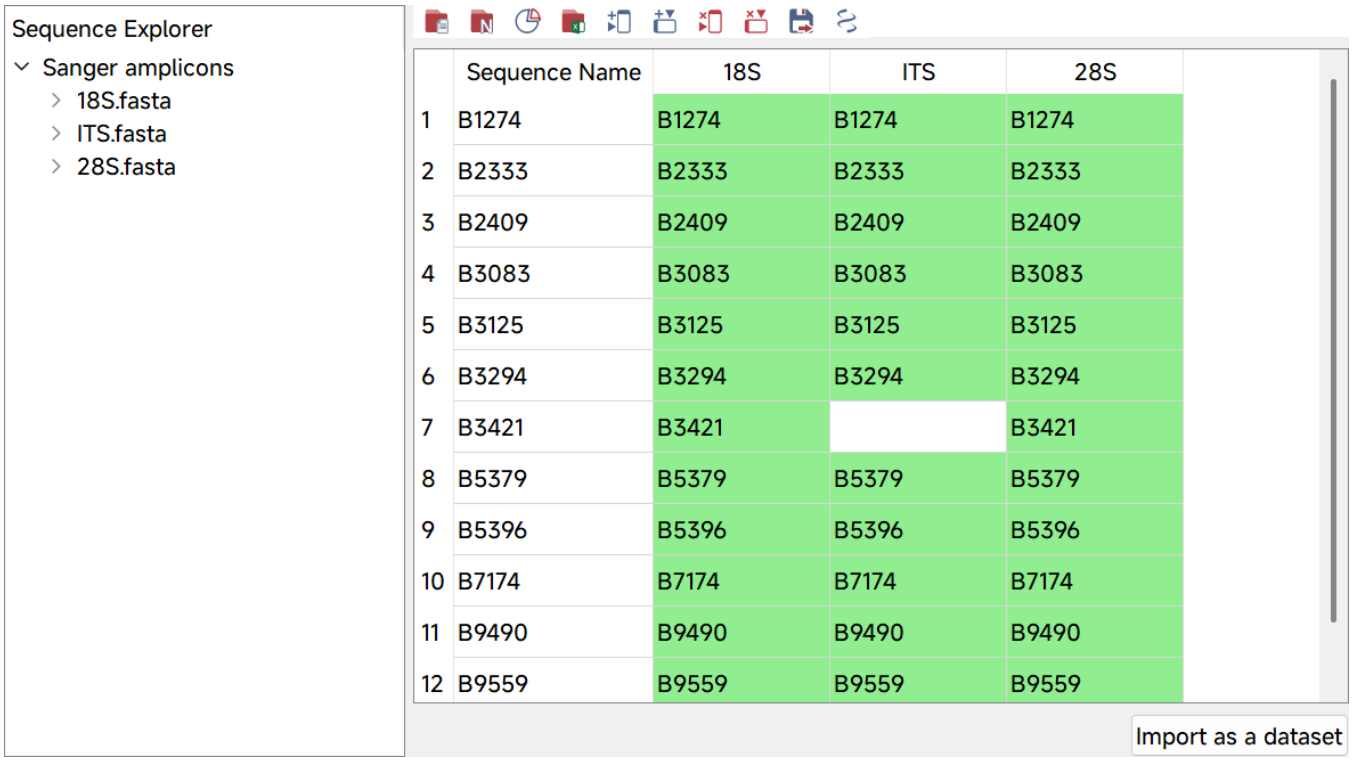


Figure 4.2: A 12-taxon, three-locus Sanger matrix. The empty ITS cell for `B3421` is retained; SeqMatrix writes it as `?` in the concatenated matrix. Green cells are sequences used once.

Amino-acid export requires CDS features to carry the `/translation` qualifier, which manually curated GenBank files may lack. Nucleotide export skips IDs containing `_protein`. SeqDBG computes adjacency within each genome, so a missing gene lowers coverage; gene-order similarity is not an orthology call, so verify one-to-one correspondence before concatenation.

4.7 Sequence Concatenator

The Sequence Concatenator is a standalone sidebar plugin in the Dataset category. It joins several per-gene alignments into one supermatrix and writes the matching NEXUS `charset` partition file, filling taxa that are missing from a gene with `?`.

1. Click **Add Files** and select the per-gene alignments (FASTA or PHYLIP). They appear in a table with their sequence count and length. The concatenation order is the order in this table; use **Remove Selected** and **Clear All** to manage it.
2. Set the **Concatenated FASTA** output path.
3. For the NEXUS partition, choose **Show Text (Default)** to preview the `charset` block in a pop-up, or **To File** to write it to disk.
4. Click **Concatenate Sequences**. After it finishes, **Display Overview Figure** opens a presence/absence heatmap of taxa × genes with a retention-per-gene row.

Taxon matching relies on an exact match of the **full FASTA header** (the string after `>`, excluding the newline). Ensure headers are identical across all input files. Missing taxa are represented by `?`, not by `-` or `N`. While PHYLIP files are selectable in the dialog, they are parsed internally as FASTA; convert PHYLIP to FASTA with consistent headers for reliable results. Every input must already be aligned and rectangular; the concatenator does not align or trim.

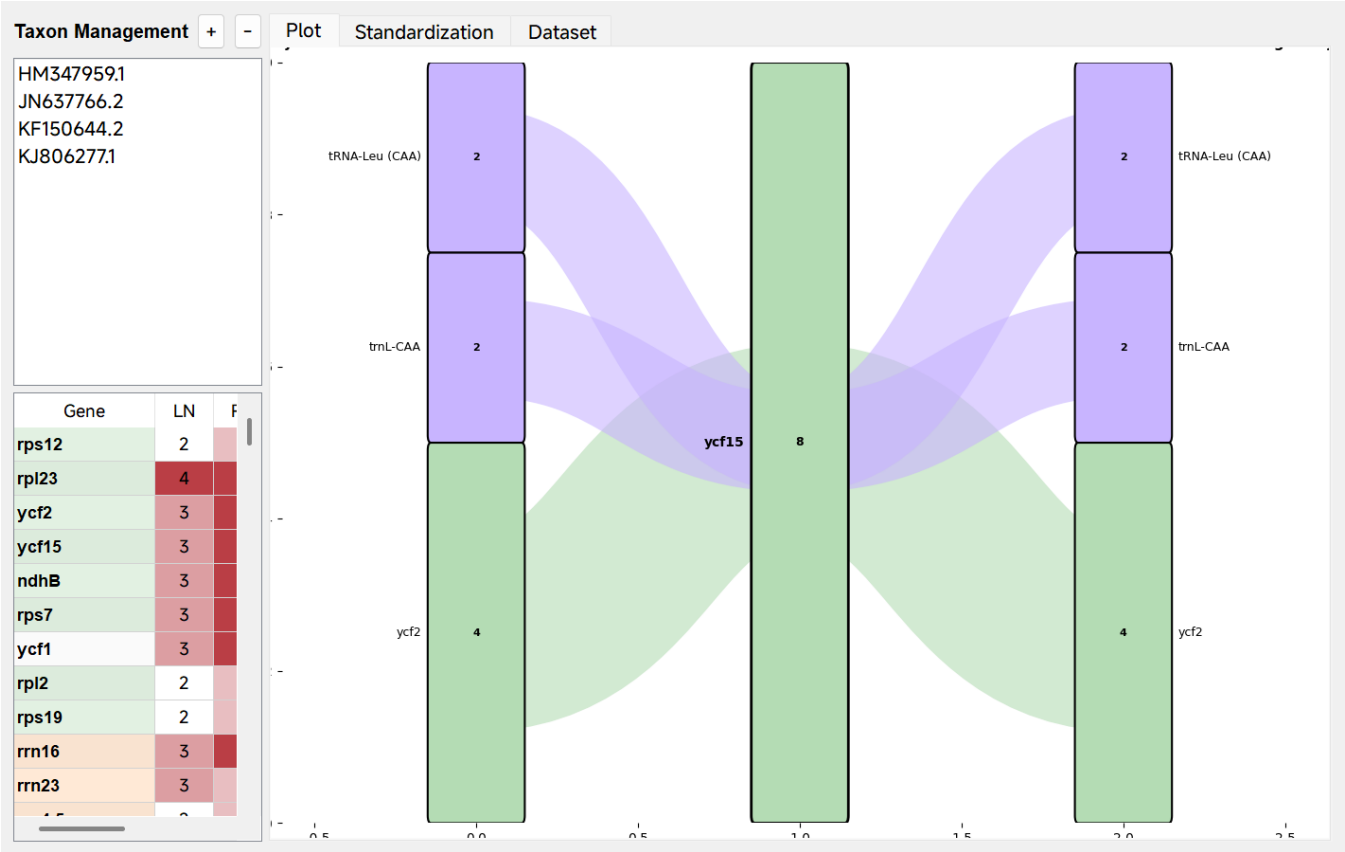


Figure 4.3: SeqDBG for four annotated chloroplast genomes. The Gene Statistics table is on the left; the selected gene and its left/right neighbours are shown in the Sankey plot.

4.8 TreeMerger

TreeMerger is a standalone sidebar plugin in the Dataset category. It pools multiple Newick gene-tree files into one file with one tree per line, which is the input format for ASTRAL and other supertree tools.

1. **Add Files** and select the gene-tree files (`*.nwk`, `*.newick`, `*.tre`, `*.tree`).
2. Set the output file path.
3. Click **Merge Trees**.

Merging is line-by-line concatenation. Each input tree must occupy a single line; a Newick string broken across lines will be copied as separate lines and will corrupt the output. Taxon labels must be consistent across all gene trees (ASTRAL fails on label mismatches). No re-rooting, de-duplication or relabelling is performed. ASTRAL treats gene trees as unrooted, so the rooting of the inputs is irrelevant, but the topology and labels are not validated before merging.

4.9 One-Step OrthoFinder for organelle genomes

WORKSPACE → Dataset → One-Step OrthoFinder (for Organelle Genomes) turns annotated genomes into a workspace dataset of single-copy orthogroups. It is intended for organelle or bacterial annotations where orthology is largely one-to-one and the user wants a ready-to-align dataset without running OrthoFinder by hand.

The pipeline is built on OrthoFinder [1] and uses a three-step panel.

Input. Select one of three source types:

- a folder of GBFF/GB/GBK files, one file per organelle genome;
- a single multi-record GenBank file, where each record becomes a separate taxon;
- a folder of proteome FASTA files.

The same panel sets the genetic code, pseudogene handling, internal-stop policy and output folder. GenBank input is parsed into CDS features, translated to protein, and written as `proteomes/<species>.faa`, `cds/<species>.fna` and `cds_map.tsv`.

OrthoFinder. Set the OrthoFinder executable, the BLAST+/MCL directory, the thread count, and the run mode: full pipeline, parse only, or reuse existing results. The runtime check reports any missing executable by name before the run starts.

Review. Shows the selected input, output, runtime and run mode before the pipeline starts.

The output tab shows summary cards (species, CDS, orthogroups, single-copy orthogroups) and a table of single-copy orthogroups. For each single-copy orthogroup the plugin writes a protein FASTA, an in-frame CDS FASTA and a mapping table, so the CDS set can be passed directly to codon-aware alignment or selection analyses. The pipeline runs in a background thread and streams the OrthoFinder log to the Console tab; it is not modal. The plugin cannot be closed while the pipeline is running.

The plugin also exposes an agent-facing action set (`inspect_options`, `modify_options`, `validate`, `run`, `stop`, `inspect_status`, `inspect_console`, `preview_results`, `export_to_workspace`); the export action is idempotent, so call it again after a completed run rather than duplicating the dataset manually.

Chapter 5

Alignment

5.1 Common aligner behaviour

MAFFT, MUSCLE 5 and Clustal Omega accept DNA, RNA or protein sequences. MACSE 2 is for protein-coding nucleotide sequences and aligns at the codon level. All four support batch input, and all four share the same input/output group: **Browse Files...** (or paste into the **Sequence** text box) and a **Save to** choice. The default output suffixes are:

Tool	Default suffix	Output
MAFFT	`_mafft`	FASTA alignment (or Clustal if requested)
MUSCLE 5	`_m5`	FASTA alignment
Clustal Omega	`_clustal`	FASTA alignment
MACSE 2	`_macse_NT`, `_macse_AA`	Nucleotide/codon and translated amino-acid alignments
TrimAl	`_trimal`	FASTA (or the selected format)
GBlocks	`_gblocks`	Block-selected alignment

Batch mode runs one process per input file. MAFFT, MUSCLE and Clustal write their output to the path chosen by the plugin; the suffix is a GUI convention, not an external option. After a run, inspect the first few sequences in the Output Preview and check the Console for warnings about duplicate names or empty records.

5.2 MAFFT

ALIGN → **Align by...** → **MAFFT** wraps MAFFT 7 [2]. In **Input / Output**, browse for one or more FASTA files or paste sequences. The default suffix is `_mafft`.

In **Basic Parameters**, choose an alignment mode:

Mode	Behaviour and typical use
Auto (`--auto`)	Default. MAFFT chooses an appropriate strategy from the input size and similarity. Good first choice for most data.
High speed	Fast progressive alignment for large, similar datasets.
High accuracy, local pair (`--localpair`)	Iterative refinement with a local alignment scoring scheme; slower, suitable for divergent sequences with local homology.
High accuracy, genaf pair (`--genafpair`)	Iterative refinement with a generalized affine scoring scheme; useful for sequences with long gaps.
High accuracy, global pair (`--globalpair`)	Iterative refinement with a global scoring scheme; suitable for similar-length, globally homologous sequences.
Fast (`--retree`)	Quick progressive alignment; use when speed matters more than precision.

The three high-accuracy modes add `--maxiterate 1000`. Set **Threads** to `-1` for automatic detection.

Advanced Parameters... exposes gap penalties (`--op`, default 1.53; `--ep`, default 0.0), maximum iterations, Clustal-format output, sequence reordering, and the strand option.

5.2.1 Automatic strand correction

Sequencing reads are sometimes returned in the reverse orientation, which ruins an alignment. In the advanced dialog, **Output Options** → **Direction adjustment** lets MAFFT detect and reverse-complement such reads so that every sequence ends up on the same strand:

- **No adjustment** (default);
- **Adjust direction** (`--adjustdirection`) — fast;
- **Adjust direction accurately** (`--adjustdirectionaccurately`) — a more exhaustive, slower check.

The option is intended for nucleotide data and needs at least two sequences. The accurate variant is worth the extra time on divergent data. Use it whenever strand is not guaranteed, for example low-throughput Sanger reads assembled without orientation control; leave it off for protein alignments.

MAFFT marks reverse-complemented records with an `_R_` prefix. The plugin strips that prefix from output headers so downstream taxon matching is not broken, while the sequences themselves remain reverse-complemented. Do not re-add `_R_` when constructing taxon labels.

Tip

If an alignment contains a few sequences that are obviously reversed, run MAFFT once with **Adjust direction accurately** and compare the output length and gap pattern with the uncorrected run. A large reduction in terminal gaps usually means the correction was needed.

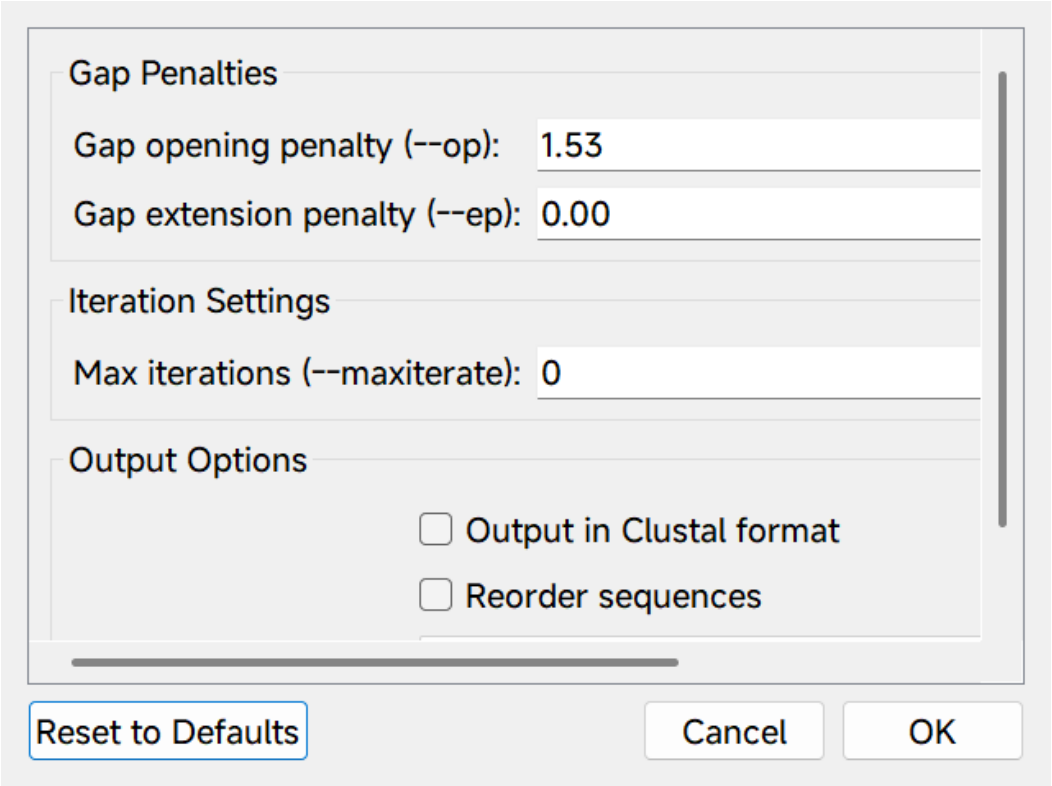


Figure 5.1: MAFFT advanced parameters with **Direction adjustment** set to `--adjustdirection`. Use this for nucleotide reads whose orientation is not guaranteed; leave it off for protein alignments.

5.3 MUSCLE 5

ALIGN → Align by... → Muscle 5 wraps MUSCLE 5 [3]. The default suffix is `_m5`. In **Parameters**, choose:

- **Standard alignment** (`-align`) — default, suitable for ordinary datasets;
- **Large dataset** (`-super5`) — for thousands of sequences.

MUSCLE 5 is extremely fast on very large numbers of sequences but can run out of memory on extremely long sequences. If it fails on long alignments, switch to MAFFT. The two modes are alternatives, not tuning flags; do not mix them within one comparison.

5.4 Clustal Omega

ALIGN → Align by... → Clustal Omega wraps Clustal Omega 1.2.2 [4]. The default suffix is `_clustal`. In **Basic Parameters**:

- **Auto options** (`--auto`) is on by default; it chooses the guide tree and HMM settings automatically.
- **Sequence type**: Auto, Protein, RNA or DNA.
- **Threads**: default 4.
- **Output order**: input order or tree order (`--outorder=tree-order`).

Clustal Omega is a good choice for large protein alignments and for users who want a familiar, fast progressive aligner. Turning **Auto** off allows manual guide-tree or HMM control, but those options only matter when Auto is off. The output is always FASTA.

5.5 MACSE 2

ALIGN → Align by... → MACSE aligns coding sequences at the codon level [5], correctly placing frameshifts and premature stop codons. Use it for CDS rather than generic nucleotide alignments.
Choose the **Mode**:

- **Align Sequences** (default) — input is unaligned CDS;
- **Refine Alignment** — input is an existing codon alignment.

Basic Parameters include the **Genetic Code** (default 1, Standard), the amino-acid alphabet (default `SE_B_8`), the score matrix (BLOSUM62, VTML200_BIS or VTML240) and the number of optimization cycles (default 2). Pick the correct NCBI genetic code for your organism; a wrong table produces many spurious stop codons.
Post-processing options are enabled by default:

- **Mask exceptional characters** rewrites MACSE's `!` characters to `?`.
- **Mask terminal stop codon** replaces a terminal stop codon with `???` in the nucleotide alignment.
- **Drop all-gap codon columns** removes codon columns in which every sequence is `!`, `-` or `?`.

These are applied by the plugin after MACSE, not by MACSE flags. Fine gap and frameshift costs go in **Advanced Parameters...** as free-text extra command-line flags; they are passed through without validation.
Run MACSE produces two files: a nucleotide/codon alignment with suffix `_macse_NT` and a translated alignment with suffix `_macse_AA`. The **Preferred result** combo controls which one is selected for import; both files are kept.

Warning

A codon alignment with many `?` or `!` columns is a warning about the input, not a success. Check the genetic code and inspect the first few codons before using the alignment for selection analysis.

5.6 TrimAl

ALIGN → Trim Alignment by... → Trimal removes unreliable columns from an alignment [6] and shows an HTML trimming report afterwards. Input formats are FASTA, CLUSTAL, PHYLIP and NEXUS; the default suffix is `_trimal`.
The main choice is the **Trimming mode**:

Mode	Behaviour
Automated 1 (`-automated1`)	Default. Chooses among the automated strategies from the alignment's characteristics; optimized for ML tree reconstruction. Recommended for most data.
Gappyout (`-gappyout`)	Uses gap statistics only; the least aggressive automated mode.
Strict (`-strict`)	Gappyout plus an automatically selected similarity cutoff; trims more than gappyout.
Strict Plus (`-strictplus`)	Stricter still; designed for neighbour-joining reconstruction.

The **Output format** is FASTA by default; CLUSTAL, NEXUS, PHYLIP, MEGA and NBRF are also available. **Advanced Parameters...** exposes the gap threshold (`-gt`), similarity threshold (`-st`), consistency threshold (`-ct`), minimum positions (`-cons`), window sizes (`-w`, `-gw`, `-sw`, `-cw`), residue and sequence overlap, and the switches `-nogaps`, `-noallgaps` and `-complementary`.

- `-nogaps` removes any column containing a gap.
- `-noallgaps` removes only columns composed entirely of gaps.
- `-complementary` writes the columns that were removed, which is useful for checking what the filter discarded.

Thresholds are emitted only when greater than zero; zero means "use the mode's internal default". `-strict` and `-automated1` are alternatives, not cumulative filters.

5.7 GBlocks

ALIGN → Trim Alignment by... → GBlocks selects conserved blocks and removes divergent or gap-rich columns [7]. Input is FASTA or NBRF/PIR; the default suffix is `_gblocks`.
GBlocks Parameters:

Option	Meaning	Default
`-t`	Sequence type: Auto, Protein, DNA or Codons	Auto
`-b1`	Minimum number of sequences for a conserved position	0 (internal default)
`-b2`	Minimum number of sequences for a flank position	0 (internal default)
`-b3`	Maximum number of contiguous non-conserved positions	8
`-b4`	Minimum length of a block	10
`-b5`	Allowed gap positions: None, With Half or All	None
`-b6`	Use a similarity matrix (proteins)	on

A value of 0 for `-b1` or `-b2` means "use GBlocks' internal default", not "allow zero sequences". GBlocks writes a block-selected alignment, not a tree. It is more conservative than TrimAl's automated modes; use it when you want explicit control over block criteria, and TrimAl when you want a fast, data-driven trim.

Chapter 6

Model selection

6.1 ModelFinder

MODEL → Find Best Substitution Model (ML) runs IQ-TREE ModelFinder [8] and imports the best model as a workspace `model` object. The core command is `iqtree3 -s <alignment> -m TESTONLY <options>`.
ModelFinder Parameters:

- **Sequence type**: AUTO, DNA or PROT. AUTO omits `-st`.
- **Criterion**: BIC (default), AICc or AIC.
- **Threads**: `-nt`; omitted when 1 or automatic.
- **Model set**: the plugin searches all models (`-mset ALL`).

The output is a ranked model table. Import the model object and select it before opening IQ-TREE or RAxML-NG; the host applies the model when the naming is compatible.

6.1.1 Partition mode

Tick **Enable Partition Mode** and open **Configure Partitions** for multi-locus data. The partition mode determines how branch lengths are shared:

Mode	Flag	Meaning
Edge-linked Proportional	`-p`	One shared topology; branch lengths are proportional across partitions. Fewest parameters; recommended for most multi-locus datasets.
Edge-linked Equal	`-q`	Shared topology and equal branch lengths across partitions. The most restrictive option; assumes homogeneous rates.
Edge-unlinked	`-Q`	Shared topology but independent branch lengths per partition. For heterogeneous rates.
Topo-unlinked	`-S`	Each partition is analysed on its own tree. This is not a combined analysis. MrBayes supports only edge-linked and edge-unlinked schemes and coerces the rest.

Optionally enable **rcluster** for fast partition merging. Define each partition's name and range, or import a NEXUS `charset` or RAxML partition file. For protein-coding genes, split codon positions by entering the position ranges (for example `1-300\3`, `2-300\3`, `3-300\3`).
In partition mode the output includes an `.iqtree` report and a `.best_scheme.nex` file. The Output Preview shows the ranked model table. Importing a multi-locus dataset from the workspace auto-enables partition mode and builds the supermatrix and partitions for you.

Warning

IQ-TREE 3.0.1 can abort partition merging with the message `Fixing branch lengths not supported under specified site rate model`. This occurs inside the partition-merger evaluation, not in the final tree search. RCluster takes a different path; the plugin retries once with `--rcluster 10` when the first attempt fails. If the error persists, run each partition separately or reduce the partition set.

6.2 ModelTest-NG

MODEL → Find Best Substitution Model (ModelTest-NG) evaluates a candidate model set and reports BIC, AIC and AICc in one run. It is useful when you want a second opinion or a model naming convention closer to RAxML-NG.

- **Sequence type**: DNA or Protein; Auto is detected from the first record.
- **Criterion**: chooses which block is exported as the best model; ModelTest-NG always reports all three criteria.
- **Starting tree**: ML or MP.
- **DNA schemes**: 3, 5, 7 or 11 substitution schemes.
- **Threads**.

ModelTest-NG can exit non-zero after writing a valid report; the plugin accepts the report when the `.out` file exists. Partition mode is not wired in this version; run model selection per alignment, or use ModelFinder for partitioned data. The best model is imported as a `model_table` object and can be applied to RAxML-NG; `+FQ` is normalised to `+FE` when the naming differs.

6.3 Model parameters

MODEL → Estimate Model Parameters (DNA, ML) fits a substitution model with IQ-TREE [9] and exports the fitted model (Q matrix, base frequencies and rate parameters) as a workspace `model` object. It can also parse an existing `.iqtree` report without rerunning IQ-TREE.

- **Input alignment**: one IQ-TREE process per file in batch mode.
- **Substitution model**: a named model, or `auto` (`MFP`).
- **Discrete Gamma**: `+G` or `+G{n}`.
- **Invariant sites**: `+I`.
- **Empirical frequencies**: `+F`.
- **Threads**.

The runner always appends `-redo`. `+G`, `+I` and `+F` are part of the same `-m` token, not separate options. Batch mode fits a separate model to each file, so it does not estimate a shared model across loci. The fitted model can be sent to IQ-TREE, RAxML-NG or the batch dN/dS MLE path.

Chapter 7

Distance and saturation

7.1 Maximum-likelihood distance

DISTANCE → Compute Pairwise Distances (ML) computes a maximum-likelihood pairwise distance matrix with IQ-TREE [9]. It supports full substitution models (GTR, HKY, and so on) with rate heterogeneity (`+G`, `+I`, `+R`) and frequency options (`+F`, `+FO`, `+FQ`) without the numerical overflow of simpler implementations.

- **Input**: FASTA or PHYLIP alignments.
- **Sequence type**: AUTO, DNA or AA; AUTO omits `-st`.
- **Substitution model**: `auto` omits `-m` and lets IQ-TREE use its distance default; a named model enables the `+G`, `+I`, `+R`, `+F`, `+FO` and `+FQ` modifiers.
- **Threads**.

The output is a lower-triangular PHYLIP matrix, `<alignment>.mldist`, previewed as a table. It is a distance matrix, not a tree; send it to DecentTree or another distance-based tree builder.

Limitations: the plugin cannot output standard deviations, cannot run bootstrap, and cannot compute p-distance. The `+R` modifier does not expose a category count here; use the IQ-TREE plugin when you need `+R{n}`. Batch mode writes one `.mldist` per input.

7.2 Uncorrected p-distance

DISTANCE → Compute Uncorrected Distances (p-distance) computes the observed proportion of differing sites. It is implemented in Python; no external program is called.

- **Input**: FASTA or NEXUS alignments.
- **Sequence type**: AUTO, DNA or Protein.
- **Gap treatment**: pairwise deletion or complete deletion.
- **DNA substitution rules**: all substitutions, transitions only, transversions only, or advanced rules. The advanced dialog lets you choose exactly which of the six substitution types to count.

The output is a square matrix with values from 0 to 1, exportable as `<basename>_uncorrected_distance.dist`. Pairwise deletion changes the compared length per pair; complete deletion uses one common set of sites. Do not compare p-distances computed with different gap treatments.

The p-distance is uncorrected: multiple substitutions at the same site are not accounted for, so it underestimates evolutionary distance at high divergence. Use it for a quick description of raw divergence, for saturation plots, or when a model-based distance is not identifiable; use ML distance for tree building.

7.3 Substitution saturation

DISTANCE → Calculate & Plot Substitution Saturation runs the BaCoCa bias/saturation analysis. It computes the index

$$C = \frac{\mathrm{std}(\mathrm{Ti}/\mathrm{Tv})}{\mathrm{std}(p)}$$

from pairwise transition/transversion counts and p-distances, and plots transitions and transversions against p-distance.

- **Input**: a nucleotide alignment (FASTA or NEXUS), or several alignments in batch mode.
- **Gap treatment**: pairwise deletion (default) or complete deletion. **View Method Description** shows the formula and the interpretation guide.

The output tab shows, for each alignment, a Ti/Tv-versus-p scatter (red = transitions, blue = transversions, with regression r) and a C-value bar chart across files. **Export Current Plot** saves the figure.

Interpretation: the more the Ti and Tv trend lines converge, the more transitions are saturated. A higher C indicates stronger bias. This is the BaCoCa C-value, not the Xia/DAMBE Iss test; there is no automatic "little/much saturation" verdict, and there are no universal C thresholds. Read the plot together with the C value. A saturated alignment may still be usable after trimming or with a site-heterogeneous model, so saturation is a diagnostic, not a model-selection substitute.

7.4 DecentTree

PHYLOGENY → Distance Methods (DecentTree) rebuilds a tree from a distance matrix using the scalable neighbour-joining family [10]. The input must be a distance matrix (`.mldist` or `.dist`), not an alignment. If you give it an alignment, compute ML distances first.

Algorithm	Note
BIONJ-R (Rapid BioNJ)	Default; verified on the Windows build.
BIONJ-V (Vectorized BIONJ)	Vectorised variant.
NJ-R-D (Double Precision Rapid NJ)	Rapid NJ with double precision.
ONJ-R (Rival Rapid NJ)	Rival rapid NJ.
ONJ-R-V (Rival Rapid NJ Vectorized)	Vectorised rival variant.
AUCTION (Auction Joining)	Auction-based joining.

The current Windows build avoids the classic BIONJ/NJ paths, which crash; the exposed algorithms are the verified variants. The tree is written next to the input as `<input>.nwk` and drawn in IcyTree. Batch mode runs one command per matrix file.

7.4.1 Negative branch lengths

Neighbour-joining is a least-squares algorithm, not a maximum-likelihood algorithm. When the distance matrix is not additive—for example because of sampling error, missing data, or a model that does not fit—NJ can produce negative branch lengths. They are a property of the algorithm on that matrix, not evidence that the matrix is invalid. The diagnostic is to compare the NJ tree with an ML tree on the same alignment and to inspect the distance matrix for non-additivity; if negative branches are few and short, they usually reflect noise around zero.

The DecentTree plugin does not modify branch lengths. For display, a negative length can be treated as zero; for downstream tools that reject negative lengths, apply a **zero-clamp** (replace negative values with 0) as an explicit post-processing step and record that you did so. A zero-clamp changes the branch-length interpretation but not the topology. Do not present clamped lengths as ML estimates.

Chapter 8

Phylogenetic inference

8.1 Maximum likelihood with IQ-TREE 3

PHYLOGENY → **Maximum Likelihood (ML)** → **IQ-Tree 3** infers a maximum-likelihood tree with IQ-TREE 3 [9]. It supports single alignments and partitioned datasets, model selection, ultrafast bootstrap, SH-aLRT and aBayes support.

8.1.1 Model and rate options

Set **Substitution Model** to `auto` (`-m MFP`, ModelFinder chooses the best model and then infers the tree), or choose a specific model and add rate heterogeneity:

- **Gamma** `+G{n}` with a category count;
- **FreeRate** `+R{n}` with a category count;
- **Invariable sites** `+I`;
- **Ascertainment bias** `+ASC`.

State Frequencies can be estimated (no suffix), empirical (`+F`), ML-optimized (`+FO`) or equal (`+FQ`). Example: GTR + Gamma 4 + Invariant + Empirical is `-m GTR+G4+I+F`; GTR + FreeRate 8 is `-m GTR+R8`.

8.1.2 Branch support

- **UFBoot** is on by default with 10,000 replicates (`-bb N`). Use at least 1,000; 10,000 is the recommended setting for publication.
- **SH-aLRT** (`-alrt N`, default 1,000) is recommended in combination with UFBoot.
- **aBayes** (`-abayes`) is a fast Bayesian approximation that can be added to either.

8.1.3 Partition mode

For multi-locus data, tick **Enable Partition Mode** and open **Configure Partitions**. The partition modes are the same as in ModelFinder: edge-linked proportional (`-p`), edge-linked equal (`-q`), edge-unlinked (`-Q`) and topo-unlinked (`-S`). Bootstrap sampling can be SITE, GENE or GENESITE; the current command builder records the choice, but the sampling flag is reserved and not yet passed. RCluster (`--rcluster N`) accelerates partition merging.

The partition thread passes `-m MFP+MERGE` when no per-partition model is set; if a `charpartition` model is present, models are read from the partition file instead.

8.1.4 Output and pitfalls

The output includes the best tree (`<input>.treefile`), the report (`<input>.iqtree`) and, in partition mode, `.best_scheme.nex`. The tree is drawn in IcyTree, and the plugin always adds `-redo`.

- ModelFinder model names use IQ-TREE syntax (`GTR+F+G4`); RAxML-NG accepts most of it but normalises `+FQ` to `+FE`.
- The supermatrix taxon set is the union across partitions; missing taxa are padded with `?`, not dropped.
- If partition merging fails with the "Fixing branch lengths" message, the plugin retries with `--rcluster 10`; otherwise reduce the partition set.
- UFBoot below 1,000 is not adequate for publication; increase the replicate count rather than the number of independent runs.

8.2 Maximum likelihood with RAxML-NG

PHYLOGENY → Maximum Likelihood (ML) → RAxML-NG infers an ML tree with RAxML-NG 1.2.2 [11]. It can run a search only, or an all-in-one search plus bootstrap analysis.

Preconditions: an aligned FASTA/PHYLIP file with at least four sequences, and a RAxML-NG model string such as `GTR+G4` or `LG+G4`. A green model object from ModelFinder or ModelTest-NG is applied automatically when the naming is compatible.

GUI control	CLI option	Notes
File input	`--msa`	One process per selected file
Model	`--model`	`+FQ` normalised to `+FE`
Threads	`--threads`	
Bootstrap replicates	`--bs-trees`	Only with `--all`; 0 means no bootstrap unless autoMRE is enabled
autoMRE convergence	`--bs-trees autoMRE{N}`	N is the replicate limit; 1,000 when the spin box is 0
Starting trees	`--tree`	Empty/default omits the flag; `pars{10}`, `rand{10}`, `rand{1}` accepted
Outgroup	`--outgroup`	Comma-separated taxa; drawing/annotation option, not an inference constraint
(fixed)	`--search` or `--all`	`--all` when bootstrap or autoMRE is active

Outputs are the best tree (`*.raxml.bestTree`), the support tree (`*.raxml.support`, with `--all`), the bootstrap tree set (`*.raxml.bootstraps`), the best model (`*.raxml.bestModel`) and the log (`*.raxml.log`).

Warning

The Windows MSYS build strips braces in `--tree`; the plugin escapes them internally, so do not pass already-escaped braces. RAxML-NG treats `--outgroup` as a drawing option, not an inference constraint. Bootstrap runs are much slower; start with a search-only run.

8.3 Bayesian inference with MrBayes

PHYLOGENY → Bayesian Inference (BI) → MrBayes3-MPI-BEAGLE runs MrBayes 3.2.7a [12] under MPI (one process per chain) and the BEAGLE library for GPU/CPU-accelerated likelihoods. A persistent workspace is required: Bayesian runs are long (often hours to days) and are checkpointed so that interrupted runs can be resumed. Intermediate chain states are written to the workspace temporary directory.

8.3.1 Input and model

Browse a PHYLIP or NEXUS alignment. Tick **Run MrBayes Data Block directly if present** to execute an existing `MRBAYES` block as-is; otherwise the plugin generates a block from the GUI settings. Non-NEXUS inputs are converted to interleaved NEXUS.

Set **Type** (DNA or Protein), **DNA substitution number** (1 = JC, 2 = HKY/K2P, 6 = GTR, passed as `lset nst=N`), **State frequencies** (estimated Dirichlet, fixed equal or empirical), **Rate heterogeneity** (Equal, Gamma, InvGamma, PropInv, Lognormal or Adgamma), **Gamma categories** (default 4) and the **Protein model** (fixed or mixed).

8.3.2 Partitions, MPI and BEAGLE

Enable Partition Mode → Configure Partitions... sets up per-partition models and rates for multi-locus data. MrBayes supports only edge-linked and edge-unlinked schemes; topo-unlinked partitions are coerced. Importing a partitioned dataset sets this up automatically.

MPI & BEAGLE Settings... keeps **Use MPI** and **Use BEAGLE** on by default. Choose the BEAGLE device (CPU or GPU), precision (Double or Single) and scaling (Dynamic or Always). MPI uses (Runs × Chains) processors; do not request more processes than the machine can supply.

8.3.3 MCMC and convergence

Default MCMC settings are 1,000,000 generations, sampling every 1,000 generations, 2 runs and 4 chains, with burn-in as a fraction (default 0.25) or as a fixed number of states. The consensus type is majority-rule, half-compatible or all-compatible; the format is FigTree or simple.

Run MrBayes. The consensus tree (`.con.tre`) is drawn in IcyTree; the `.p` parameter chains can be sent to MiniTracer. There is no automatic stop rule, so decide burn-in and run length from the diagnostics:

every parameter should reach ESS $\geq$ 200 in MiniTracer, the potential scale reduction factors (PSRF) from `sumt` should be close to 1.0, and the traces should show no trend with the two independent runs in agreement. If not, increase the number of generations and re-run. The plugin appends `sump` and `sumt` after `mcmc`; without `mcmc;` the block would only read the data.

8.4 Bayesian inference with PhyloBayes

PHYLOGENY → Bayesian Inference (BI) → PhyloBayes-MPI runs PhyloBayes-MPI 1.9 [13] under MPI. PhyloBayes specialises in site-heterogeneous infinite-mixture (CAT) models, which reduce long-branch attraction. A persistent workspace is required.

Input: a PHYLIP alignment (convert FASTA first), or a chain file to continue. PhyloBayes forces a global mixture model, so partition information is not imported.

Choose **site heterogeneity** (Infinite CAT, `-cat`, default, or finite nCAT, `-ncat N`), **rate heterogeneity** (gamma with categories, `-dgam N`, default 4) and the **substitution model** (GTR default, or Poisson, JTT, WAG or LG for proteins).

MCMC and run settings: number of MPI parallels, number of parallel chains (1–4; run at least 2 independent chains for convergence), MCMC generations (default 1,000; set far higher for real runs; −1 means run indefinitely) and sampling frequency. Gamma and CAT can be combined; both are on by default. The generation count and sampling frequency are emitted as a single `-x FREQ GEN` pair.

Run PhyloBayes-MPI, then open the **Compare & Consensus** sub-tab and run `bpcomp` with a burn-in and a cutoff (default 0.50, i.e. 50% majority-rule) to build the consensus and the bipartition list. Check convergence with `bpcomp`'s `maxdiff`; aim for a value below 0.1 between independent chains before trusting the tree.

8.5 Maximum parsimony with MPBoot

PHYLOGENY → Maximum Parsimony (MPBoot) infers a maximum-parsimony tree with MPBoot's ultrafast bootstrap and consensus reconstruction [14].

Basic Parameters:

- **Sequence type**: AUTO, DNA, AA, CODON, BIN or MORPH.
- **Threads**.
- **Ultrafast bootstrap (UFBoot)**: on by default, 1,000 replicates.
- **Consensus threshold**: default 0.50; 0 produces an extended consensus.
- **Burn-in**: omitted at 0.

Advanced Parameters... controls the tree search: initial, best and candidate trees; NNI versus SPR; ratchet iterations, weight and percentage; SPR radius; candidate cutoff; and tree refinement. The dialog values only take effect after you accept the dialog.

Run MPBoot produces `<input>.treefile` (best tree) and `<input>.contree` (consensus), drawn in IcyTree. `-con` is always emitted, so a consensus tree is produced even when the threshold is 0. `-bb` should be at least 1,000.

8.6 MCMC diagnostics with MiniTracer

PHYLOGENY → Bayesian Inference (BI) → MCMC Diagnostics (MiniTracer) opens a lightweight convergence-diagnostics viewer in the spirit of Tracer. It reads the parameter files produced by BEAST (`.log` or `.log.txt`), MrBayes (`.p`) and PhyloBayes (`.trace`). You can also double-click a chain object in the workspace, and the chains are imported automatically.

8.6.1 Loading and burn-in

Load one or more trace files with **+ (Select MCMC Trace Files)**. MrBayes `.p` and PhyloBayes `.trace` are accepted, but they do not appear in the default filter; switch to **All Files (*)** to pick them, or open MiniTracer from a green chain object. Use **-** to remove a file and **Refresh** to reload after editing the burn-in.

Set the **Burn-in** per file as a fraction of total generations (default 0.100). Discarded states are shown greyed at reduced opacity. Burn-in is entered as a fraction, not as a generation count.

8.6.2 Statistics table and single-chain view

The statistics table on the left lists each parameter's **Mean** and **ESS**. The ESS is colour-coded per row:

- **green**: ESS $\geq$ 200, sufficient;
- **yellow**: 100 $\leq$ ESS $<$ 200, marginal;
- **red**: ESS $<$ 100, insufficient.

Single-click a parameter to draw it. The **Analyze Single Chain** tab shows the trace plot (**MCMC Trace**) beside the post-burn-in density (**Post-burnin Distribution**, a KDE plus histogram), with a summary table reporting the number of samples, mean, range, 95% HPD interval and ESS.

8.6.3 Two-chain comparison and PSRF

To compare independent runs, select exactly two files and open the **Compare 2 Chains** tab. The tab draws a Chain 1 versus Chain 2 scatter with a red dashed $y = x$ reference line. The table reports each chain's sample count and mean, the correlation coefficient **R**, and a **PSRF (R-hat)** row.

The PSRF is the Gelman–Rubin potential scale reduction factor computed from the two post-burn-in chains. Values near 1 indicate that the between-chain variance is small relative to the within-chain variance, i.e. that the chains agree. The value is also shown in the plot title (`Chain Comparison (PSRF = ...)`). A value of `inf` means the two chains are constant but different; `N/A` means the statistic is undefined, usually because there are fewer than two paired samples.

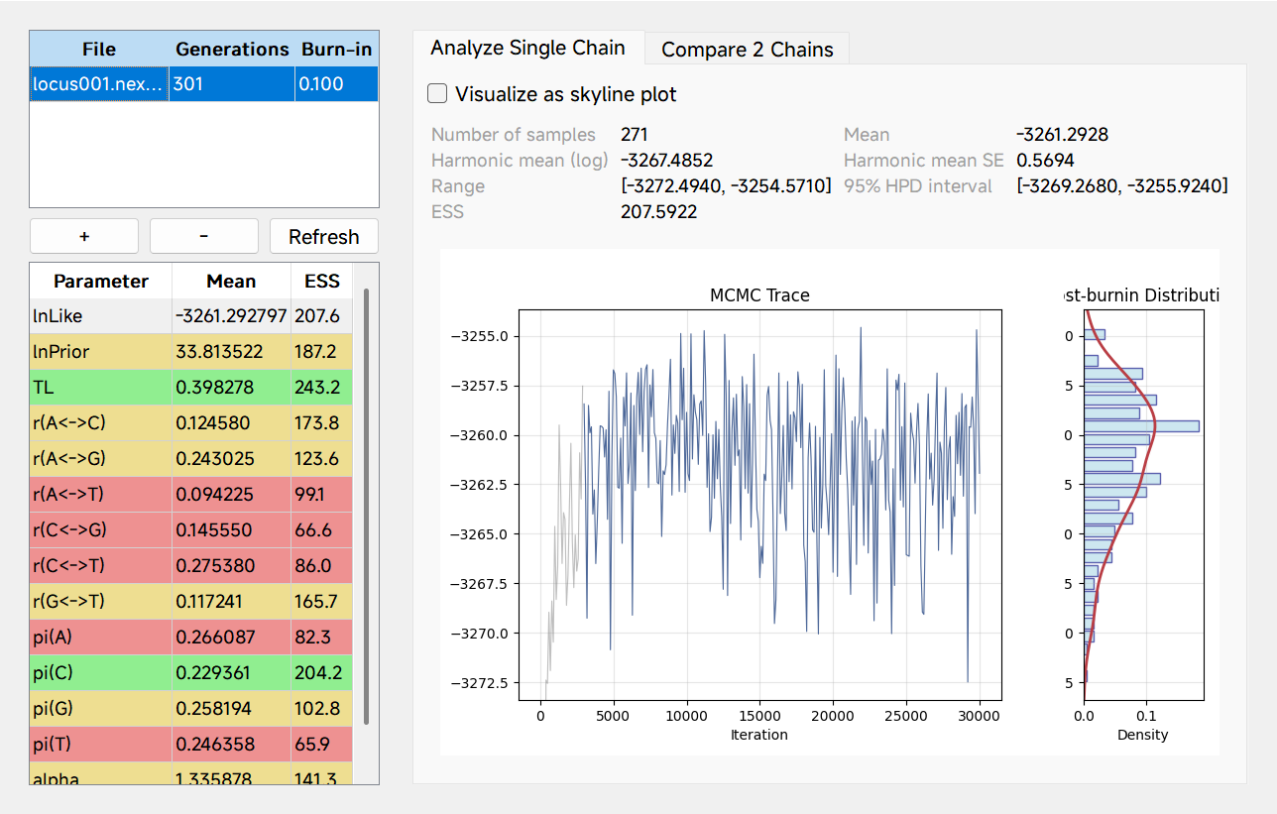


Figure 8.1: MiniTracer single-chain view for `lnLike`: post-burn-in trace, marginal density, ESS, 95% HPD and the per-parameter statistics table. Green, yellow and red rows mark ESS ≥ 200, 100–200 and < 100.

Tip

Treat an analysis as converged only when every parameter has ESS ≥ 200 and the two independent runs agree (points near the $y = x$ line, high R, PSRF near 1). If a parameter has a low ESS but a stable trace, increase sampling or thin the chain; if the trace has a trend, increase burn-in or run length.

MiniTracer currently does not export plots or tables; take a screenshot if you need a figure, or switch to Tracer for exportable graphs. The harmonic-mean panel, where present, is descriptive and is not a substitute for a proper marginal-likelihood comparison.

8.7 Tree comparison with Robinson–Foulds distances

PHYLOGENY → Tree Comparison → Pairwise Robinson–Foulds compares a set of trees with IQ-TREE 3 [9]. The input is a Newick file with one tree per line, or two or more phylogeny/tree-set objects sent from **Selected**. IQ-TREE computes the all-to-all split distance with `-t trees --tree-dist-all` and writes `rf.rfdist`; the plugin parses that matrix and maps the `Tree0`, `Tree1`, … rows back to the workspace object names.

The **Overview** tab is a box plot of all pairwise RF distances with the individual comparisons overlaid. When the input contains one or more **tree-set** objects, the plot separates *within tree set* pairs from *between tree sets* pairs; pairs that involve a plain phylogeny are shown separately as *between objects*. This makes the spread of a single posterior or bootstrap sample directly comparable with the divergence between independent runs. The **RF matrix** tab shows the lower triangle with the diagonal and upper triangle masked. The **Pairs** tab lists every comparison, its category, and, for four or more taxa, the normalised distance $\mathrm{RF}/[2(n-3)]$, the maximum RF between two fully resolved unrooted trees.

IQ-TREE treats a tree whose root has two children as rooted and otherwise as unrooted; mixing the two makes `--tree-dist-all` report a sentinel distance. The plugin therefore parses each line with Biopython and rewrites

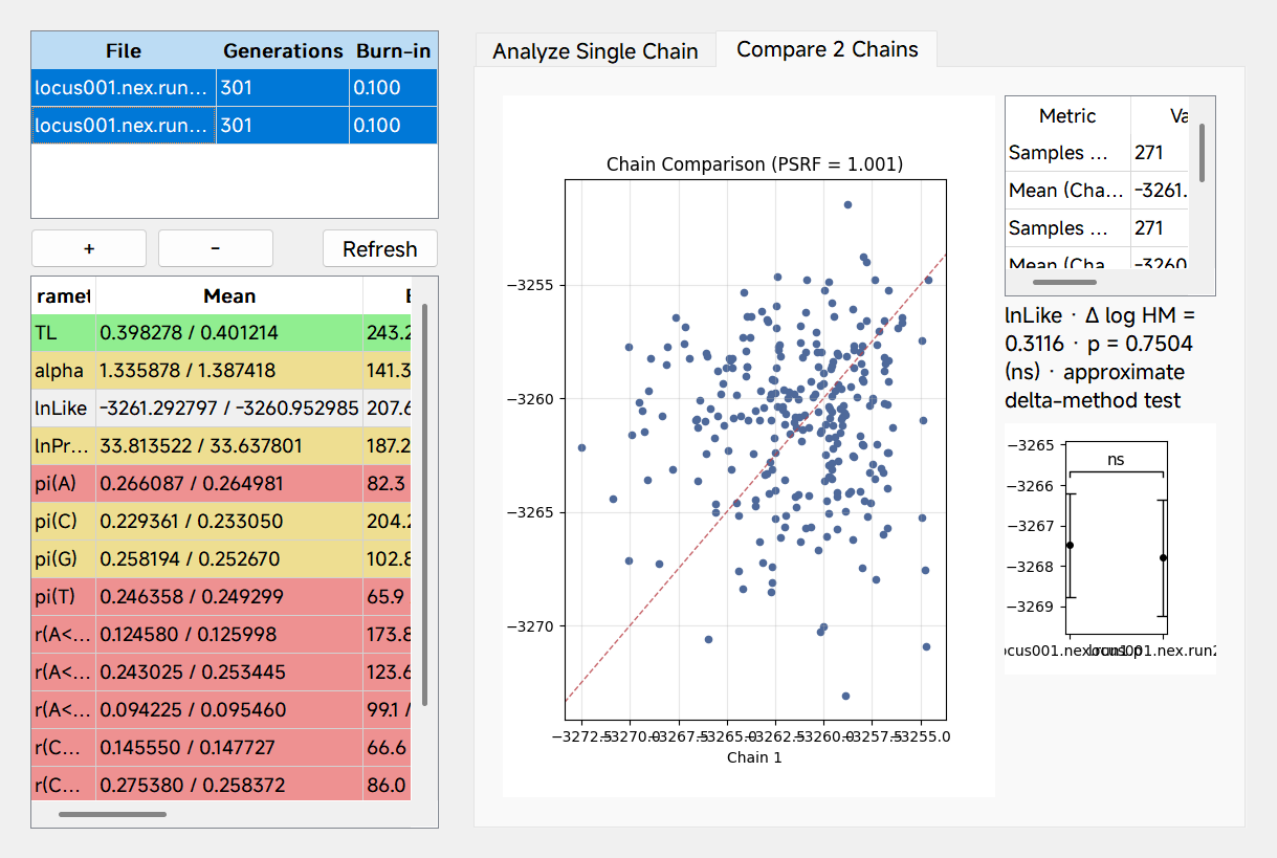


Figure 8.2: MiniTracer two-chain comparison. Points near the red $y = x$ line indicate agreement; the table reports sample counts, means, the correlation coefficient and the Gelman–Rubin PSRF (R-hat), also shown in the plot title.

rooted trees as unrooted topologies before running IQ-TREE. RF is an unnormalised split distance and is not a substitution distance; trees with different taxon sets are handled by IQ-TREE but are less directly comparable. The pairwise and matrix tables export as TSV, and both figures can be saved as PNG.

Chapter 9

Species tree and concordance

9.1 ASTRAL-IV and wASTRAL

The **SPECIES TREE** menu exposes ASTRAL-IV, wASTRAL, CASTER-site, TreeRecs, gene and site concordance factors, concordance diagnostics and BEST/MrBayes MSC.

ASTRAL infers a species tree from gene trees under the multi-species coalescent [15]. The workbench wraps the ASTER executables:

- **ASTRAL-IV** (`astral4`) — standard quartet-based species-tree inference with CASTLES-II branch lengths;
- **wASTRAL** (`wastral`) — weighted ASTRAL; weights quartets by gene-tree support and branch length, down-weighting poorly supported quartets and long branches, and reports coalescent-unit branch lengths. **ASTRAL-Hybrid** (`astral-hybrid`) is a synonym/preset of wASTRAL, not a separate method; both use support values and branch lengths.

ASTRAL-Pro 3, which handles gene trees with multiple individuals per species, is not exposed in this version. The bundled ASTER executables report version 1.24; ASTER2 is not part of the current binary bundle.

1. Provide one Newick file with one gene tree per line. Opening ASTRAL from **Selected** with phylogeny/tree-set objects preloads their trees; **Import from Selected TreeSet** is the fallback and also accepts plain phylogenies.
2. Choose ASTRAL-IV, wASTRAL or ASTRAL-Hybrid.
3. Set threads, the support option (0 = none, 1 = length and support, 2 = detailed, 3 = `freqQuad.csv`) and an optional output path.
4. Advanced options cover constraint and guide trees, rounds, subsample rounds, a gene-to-taxon map, root taxon, subsample proportion, seed, Yule lambda, repeat downweighting and verbosity.
5. Run and send the resulting tree to the workspace.

ASTRAL-IV and CASTER-site default to `SULength`, which includes terminal branch lengths and is the most readable option in tree viewers. wASTRAL reports coalescent units and intentionally leaves terminal branch lengths empty; the plugin adds display-only terminal lengths of 10^{-5} so viewers that reject missing tip lengths can open the tree. The plugin keeps the tree written through `-o` instead of overwriting it with stdout logs.

Constraint and guide trees must be binary Newick trees. `--length` and `--genelength` are ASTRAL-IV options and must not be passed to wASTRAL or ASTRAL-Hybrid. ASTER's weighted parsers call `std::stod` on node labels; IQ-TREE composite labels such as `/1/100` can make them abort. The plugin normalises composite labels before weighted runs, keeping the first numeric field.

9.2 CASTER-site

SPECIES TREE → CASTER-site infers a species tree directly from a sequence alignment by site-based quartet sampling [16]. No gene trees are needed, and the method is very fast when the alignment is large enough.

- **Input**: FASTA, PHYLIP, or a file list.
- **Format**: auto, fasta, list or phylip. `list` expects a file containing paths to per-locus alignments, not a FASTA alignment.
- **Threads**, **support option** and optional output path.
- **Advanced**: subsample rounds, constraint and guide trees, rounds, gene-to-taxon map, root taxon, subsample proportion, seed, chunk size, ambiguity handling (0 = treat ambiguity codes as N; 1 = diploid unphased sites), branch lengths and scoring-only mode.

Warning

Ensure the alignment or supermatrix has enough sites. Empirically, CASTER-site needs on the order of 100 kbp (for example a chloroplast genome) to converge. With fewer informative sites, the tree may be unresolved or unstable even though the program exits successfully.

The output is a Newick tree with branch lengths. Terminal lengths may be missing and are added by the plugin for viewers. CASTER-site can be slow on large alignments; increase threads before increasing rounds.

9.3 Gene and site concordance factors

Concordance factors annotate a fixed species tree with the proportion of gene trees (gCF) or alignment sites (sCF) that support each internal split. They are not branch support values and should not be interpreted as posterior probabilities.

9.3.1 Gene concordance factors (gCF)

SPECIES TREE → Gene Concordance (gCF) takes one species tree plus one file of gene trees (one Newick tree per line). It runs IQ-TREE 3 [9] with `--gcf`, using the species tree as the starting tree (`-t`). The output is a TSV table with one row per internal bipartition. Each split is named by its **smaller taxon side**, so the representation is stable across programs.

Columns: `split`, `split_size`, `branch_id`, `gCF`, `gDF1`, `gDF2`, `gDFP`, `gN`, `branch_length`. The four gene-tree categories sum to 100%: `gCF + gDF1 + gDF2 + gDFP = 100%`. `gN` is the number of decisive gene trees; `gDFP` is polyphyly/other, not a clean NNI alternative.

The **Table** tab shows the parsed rows; the **Overview** tab shows the distribution. Send the table to the workspace as a `table` object. A gCF at or below 1/3 cannot be inverted to a positive coalescent-unit length, so do not force the transformation.

9.3.2 Site concordance factors (sCF)

SPECIES TREE → Site Concordance (sCF) takes one species tree plus one alignment. It runs IQ-TREE 3 with `--scfl` and a fixed species tree (`-te`), using likelihood-based quartet sampling. The number of quartets is set in **sCF quartets** (default 1,000). The output is a TSV table with columns `split`, `split_size`, `branch_id`, `sCF`, `sDF1`, `sDF2`, `sN` and `branch_length`.

The current workbench exposes the likelihood sCF estimator only. IQ-TREE also implements a parsimony sCF (`--scf`), which is faster but ignores the substitution model; it is not exposed here, and results from the two estimators should not be mixed. sCF depends on the alignment and substitution model: an uninformative or misaligned partition can return zero informative sites.

9.3.3 Concordance diagnostics

SPECIES TREE → Site Diagnostics (sCF + CASTER) compares likelihood sCF with CASTER-site q1/q2/q3 scores on the same fixed tree; its two Simplex tabs show sCF/sDF1/sDF2 and CASTER q1/q2/q3 as separate figures, and the CASTER panel encodes CULength as colour. **SPECIES TREE → MSC Diagnostics (gCF + ASTER CU)** compares gCF with ASTRAL-IV and wASTRAL coalescent-unit lengths. These panels are interpretive aids:

they align the same splits across methods and flag disagreements, but they do not replace a formal concordance analysis.

9.3.4 MSC diagnostics and simplex visualisation

SPECIES TREE → MSC Diagnostics (gCF + ASTER CU) is the combined view. It takes one fixed species tree, a set of gene trees and, optionally, the alignment used to estimate them. The plugin runs IQ-TREE 3 gCF, scores the same tree with ASTRAL-IV and wASTRAL, and presents the result in four tabs:

- **Table** — one row per internal split, with gCF/gDF1/gDF2/gDFP and the ASTER coalescent-unit values;
- **Overview** — the distribution of the concordance factors and the largest gCF versus the largest gDF, so discordant splits stand out;
- **Simplex** — ternary plots of the gCF and ASTRAL-IV qCF triplets, with gDFP encoded by colour for gCF and CULength for qCF;
- **Bipartition info** — a taxa × splits membership matrix, with the smaller side of each split shaded blue.

The simplex is a ternary coordinate system for the three competing quartet outcomes. For gCF, the vertices are `gCF`, `gDF1` and `gDF2`; a point near the top vertex means that most decisive gene trees support the species-tree split, and a point near a bottom vertex means that an alternative NNI split is more common. `gDFP` is not a vertex because it mixes polyphyly and other outcomes; it is shown as the point colour. The ASTRAL-IV panel uses `qCF`, `qDF1` and `qDF2` in the same way, and colours each point by its coalescent-unit length (`CULength`), so a split's concordance position and its CU length can be read together. The two panels are separate because the order of `DF1` and `DF2` can differ between the gene-tree and site-based summaries.

Each point is an *internal split*, not a gene tree. A tree with n taxa has $n - 3$ internal splits in the unrooted case, so a 10-taxon analysis has at most seven points. Increasing the number of gene trees improves the precision of each point; it does not add points. A denser simplex requires more taxa.

The bipartition matrix uses the same split ordering. Rows are taxa and columns are `S1`, `S2`, …; a blue cell means that the taxon is on the smaller side of that split. Hover a column header to see the full split. This view is useful for checking that a split is not driven by a single taxon or by a missing-data pattern.

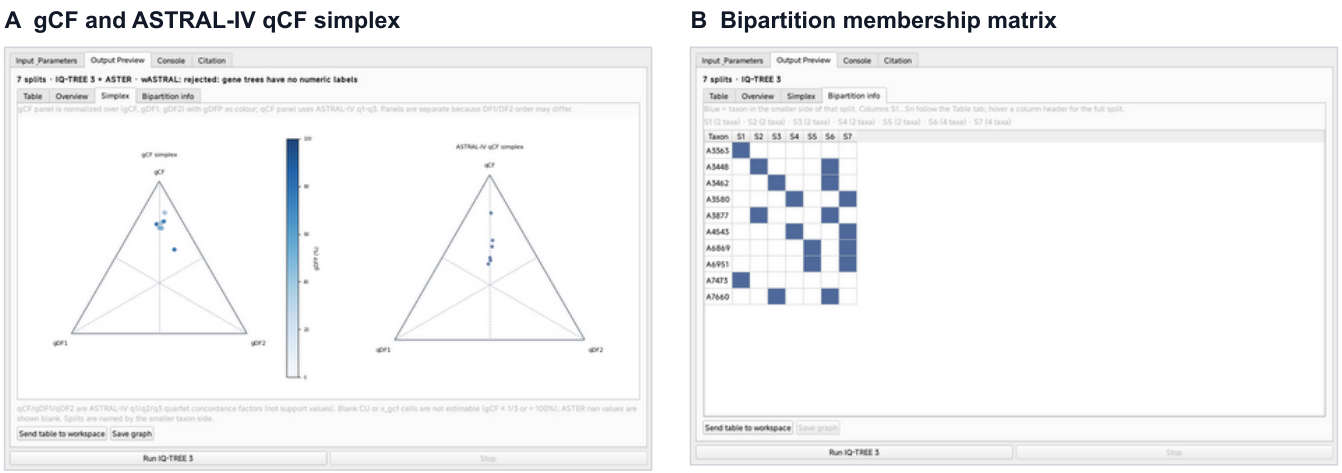


Figure 9.1: MSC diagnostics. (A) gCF and ASTRAL-IV qCF simplex views; each point is an internal split, gDFP is encoded by colour for gCF and CULength for qCF. (B) Bipartition membership matrix; rows are taxa and columns are splits, with the smaller side shaded blue.

A gCF at or below 1/3 cannot be inverted to a positive coalescent-unit length, and a gCF of 100% leaves no discordance to estimate; the plugin leaves those CU cells blank rather than printing a misleading number. ASTER may reject a wASTRAL run when the gene trees carry no numeric node labels; the diagnostic header reports this as `wASTRAL: rejected`. In that case the gCF and ASTRAL-IV qCF panels are still valid, but the wASTRAL CU

comparison is not available. The **Save graph** button writes the currently visible Overview or Simplex figure; **Send table to workspace** exports the table as a typed item.

Do not read the simplex as a posterior distribution, and do not compare the absolute positions of points from analyses with different taxon sets. Read it together with the gCF table, the bipartition matrix and the CU column.

9.4 Gene tree reconciliation with TreeRecs

SPECIES TREE → TreeRecs 1.2 reconciles a gene tree with a species tree, reports duplication and loss events, and optionally writes orthology/paralogy events.

- **Gene tree**: Newick.
- **Species tree**: Newick.
- **Duplication cost** (default 2) and **loss cost** (default 1).
- **Best root by reconciliation**: let TreeRecs choose the root.
- **Contraction threshold**: omit unless a numeric threshold is needed.
- **Orthology/paralogy events** and an optional gene-to-species mapping file.

TreeRecs runs with `-f -q` (fix root, quiet) and creates its output directory. A contraction threshold of "none" must not be passed as `-t none`; TreeRecs 1.2 aborts on it, so the plugin omits the flag unless a numeric threshold is set. Passing `-p N` or `-c _` explicitly switches separator mapping on and can break leaf names such as `A/B/C`; both flags are emitted only when changed from their defaults. For a separate mapping file, use the mapping option rather than the species-prefix convention.

9.5 BEST and MrBayes MSC

SPECIES TREE → BEST / MrBayes MSC exposes the MrBayes multi-species-coalescent analysis. The input must be a NEXUS data file. Partitions may be loaded from `charset` definitions; the species partition uses MrBayes `speciespartition` syntax.

The panel is block-oriented. It shows the canonical `MRBAYES` block, accepts a complete block back, and reports any lines that were not parsed. Check the round-trip response before running. The analysis starts MrBayes through the task queue; the consensus tree and chain files can be exported to the workspace. A complete block should contain the `mcmc` command and the species-partition directives; an incomplete block may read the data without sampling.

Chapter 10

Molecular dating

10.1 Choosing a dating route

YR-MPE offers two implemented routes to a time-scaled tree, plus a planned approximate-likelihood route through PAML MCMCTree. Use **LSD2** for a fast point estimate on a reliable fixed tree, **MrBayes dating** when the topology and dates should be inferred jointly with credible intervals, and **MCMCTree approximate** when a Bayesian relaxed clock is needed but the full MCMC is too expensive; the latter is an external workflow here.

Enter all ages in one consistent time unit (years before present, million years ago, or arbitrary time units). Mixing units is the most common dating error and is not detected by the programs.

10.2 LSD2

CLOCK → **LSD2** dates a fixed input tree given calibrations (root-to-tip regression) [17]. The current workspace tree is imported automatically.

Set the **Tree file** (Newick) and, optionally, a sequence file (used only to derive sequence length). Root the tree with **Choose root** (IcyTree) or let LSD2 **Estimate the root**. For tip dates (sampling dates, for example virus isolation dates), tick **Use tip dates** → **Configure tip dates** and, per OTU, choose a type (Point, Uniform, Upper Boundary or Lower Boundary) and an age. For node (MRCA) calibrations, press + under **Calibration points**, then **Edit** to open the MD-MRCA dialog; pick the taxa defining the clade, name it, choose a calibration type (Point, Uniform, Upper Boundary, Lower Boundary, Normal or Lognormal) and set its parameters. A live preview shows the distribution on the tree.

Optionally enable **Calculate confidence intervals**: LSD2 simulates 100 trees under a lognormal model of branch-length uncertainty, with standard deviation q (default 0.2; larger q gives wider intervals). Tip dates are not resampled. Run LSD2 to write a dated Newick tree plus a `.result` report, drawn in IcyTree.

Sequence length is always passed to LSD2: Disabled uses 1000, Manual uses the spin value, and From sequence file computes it. **Use chosen root** only takes effect if a rooted tree file is loaded; otherwise LSD2 runs with `-r a` (estimate the root). The confidence-interval option fixes `-f` at 100 bootstrap trees.

Warning

Normal and Lognormal MRCA priors are automatically collapsed to a uniform 95% HPD interval by the YR-MPE dialog, with a red warning. This is a *preprocessing choice by YR-MPE*, not a behaviour of LSD2 itself.

10.3 MrBayes molecular dating

Dating is a mode inside the MrBayes plugin [12]. Open **CLOCK** → **MrBayes3-MPI-BEAGLE** (the same plugin as Bayesian inference), tick **Enable Phylogenetic Dating**, then click **Dating Settings...**.

- **Clock Model**: Strict Clock (default), Independent Gamma Rates (IGR), Compound Poisson Process (CPP)

or Thorne–Kishino 2002 (TK02). Relaxed clocks expose a rate prior and mean.

- **Outgroup Settings**: tick the taxa that form the outgroup, for rooting.
- **Calibration Nodes**: **+ Add** → **Edit** each MRCA. Give it a node name, select its taxa, choose a **Prior Type** (Fixed Value, Uniform Distribution, Offset Exponential, Offset Gamma, Offset Lognormal or Truncated Normal) and set its parameters. Ticking a node's **Constraint** applies the calibration to that clade.

Set the usual MCMC generations, runs, chains and burn-in, then run MrBayes-MPI-BEAGLE. The dating commands are injected automatically and the `sumt` consensus is time-scaled. Per-taxon tip dating is currently wired into LSD2 only; for a tip-dated MrBayes run, prepare the command block externally.

10.4 MCMCTree approximate likelihood

MCMCTree is part of PAML 4.10.10 [18, 19]. It estimates divergence times under a Bayesian relaxed clock using a tree, an alignment and calibrations. The **approximate likelihood** mode (`usedata = 3`) replaces the exact site-pattern likelihood with a Taylor expansion around the ML estimates, using the gradient and Hessian stored in `in.BV`. It is much faster than exact MCMCTree and is appropriate for large alignments when the approximation is acceptable.

Warning

MCMCTree is *not yet wrapped by a YPhylo Workbench plugin* in version 1.2.0. The **CLOCK** menu reserves the entry, and **Import to MCMCTree** reports that the import will be wired through PD-Guide. The workflow below is the supported external route with PAML.

The external workflow:

1. Infer a rooted, branch-length tree with IQ-TREE 3 or RAxML-NG and prepare a PHYLIP alignment with the same taxon labels.
2. Write a `mcmctree.ctl` control file. The essential fields are `seqfile`, `treefile`, `outfile`, `ndata`, `usedata = 3`, `clock = 2` (independent rates) or `clock = 3` (correlated rates), `RootAge`, `model`, `alpha`, `ncatG`, `cleandata`, `BDparas`, `rgene_gamma`, `sigma2_gamma`, `burnin`, `sampfreq` and `nsample`.
3. Write a `calibration` file for the calibrated nodes. Each line gives the node, a soft or hard bound, and the age parameters; the tree file must contain the corresponding node labels.
4. Run once with `usedata = 2` (exact likelihood) to calculate the gradient and Hessian and write `in.BV`, then switch to `usedata = 3`. Inspect the `mcmc.txt` trace and summary tree for convergence and effective sample sizes.

The approximation is local: it is reliable when the ML estimates are close to the posterior mode and the alignment is not dominated by saturation or model misspecification. If the approximate and exact runs disagree, use the exact likelihood or a different clock model.

10.5 IQ2MC conversion

IQ2MC is the planned converter from an IQ-TREE analysis to MCMCTree input: it will write the PHYLIP alignment, rooted tree, `mcmctree.ctl`, `calibration` file and the `usedata = 2` run that produces `in.BV`, making the approximate workflow a one-click continuation of an IQ-TREE analysis. It is not implemented in 1.2.0; until then, follow the external workflow above.

10.6 PD-Guide

CLOCK → **PD-Guide** is a guidance layer over the dating controls. It can infer a tree with IQ-TREE, configure tip dates and node calibrations, export the calibration table, and run LSD2. Only LSD2 is wired end-to-end; the other methods report "not yet implemented". Export the table to reuse it with MrBayes or MCMCTree.

Chapter 11

Selection analysis

11.1 Codon-based Z-test: NG86 and YN00

The **SELECTION** toolbar menu opens the codon-based Z-test panel. The input is an in-frame codon alignment in FASTA format (a multiple of three nucleotides, with no internal stop codons in the analysed codons). The method menu offers two pairwise estimators.

NG86 counts synonymous and nonsynonymous sites and differences with the Nei–Gojobori method and applies the Jukes–Cantor correction. The analytic Z-test compares d_N and d_S; the variance is $\mathrm{Var}(d_N) + \mathrm{Var}(d_S)$, using the numbers of synonymous and nonsynonymous sites as the binomial denominators.

YN00 implements the Yang–Nielsen (2000) method with equal pathway weighting, following the default mode of PAML's `yn00`. Codon frequencies are estimated from the average 3×4 base composition of the pair, one transition/transversion ratio κ is estimated for the alignment from non-degenerate and fourfold-degenerate sites, and d_S and d_N are corrected with the F84 model. The standard errors of d_S and d_N come from the F84 variance, and $Z = (d_N - d_S)/\sqrt{\mathrm{SE}(d_N)^2 + \mathrm{SE}(d_S)^2}$. The implementation reproduces PAML `yn00` on the `abglobin` example to four decimal places [18].

The table reports S, N, S_d, N_d, d_S (with SE for YN00), d_N (with SE), $\omega = d_N/d_S$, Z, and the conclusion. The matrix view follows MEGA: the upper triangle contains two-sided p-values and the lower triangle contains ω. Cells with $p < 0.05$ are bold and tinted (red for $d_N > d_S$, green for $d_N < d_S$). Codons with gaps, ambiguous bases, or stops are skipped pairwise. Changes that would create a stop codon are excluded when counting synonymous sites, and mutation paths through stop codons are discarded when counting multi-step codon differences. Export writes the table or matrix as TSV or CSV.

11.2 Batch dN/dS: NG86, YN00 and IQ-TREE MLE

The batch panel accepts a folder of codon alignments or alignment items from the workspace. In *Counting* mode it applies the same NG86 or YN00 implementation and writes one row per sequence pair. In *MLE* mode it runs IQ-TREE 3 for each alignment with `--seqtype CODON` and a codon model (`-m GY` or `-m MG`), then reports the global ω and log-likelihood parsed from the IQ-TREE report. The MLE controls (threads and output folder) are enabled only in MLE mode.

Counting is a fast approximation and does not require an external binary; MLE is slower but uses a codon-substitution model. Both counting modes and MLE export a TSV table. The MLE output folder is required; picking a file instead of a folder is not supported. IQ-TREE must be configured before MLE runs. The genetic code applies to both counting and MLE paths.

11.3 CodeML and HyPhy (planned)

Site, branch and branch-site models in PAML CodeML [18] and HyPhy [20] are not implemented yet. The **SELECTION → CodeML / HyPhy (planned)** entry is disabled. For a site-model analysis today, export the codon alignment and run CodeML or HyPhy externally; the workbench can still manage the input alignment, the model report and the resulting table as workspace objects.

Chapter 12

Tree visualization

12.1 IcyTree viewer

PHYLOGENY → Tree Viewer (IcyTree) opens the embedded IcyTree viewer. It also opens automatically whenever a tree result is produced. IcyTree is an upstream web application embedded in a Qt window; YR-MPE intercepts its file downloads with its own save dialog.

Opening a tree from the workspace auto-loads the latest tree. From the menu, use IcyTree's own **File → Load from file...** or **Enter tree directly...**; accepted formats are Newick, NEXUS, PhyloXML and NeXML. NEXUS is preferred when the tree carries rich annotations, because it can preserve support values, branch lengths and metadata that plain Newick may lose.

12.2 Display, rooting and support

The IcyTree menu bar provides:

- **File** — load trees or CSV metadata; export the tree as SVG, PNG, JPEG, Newick, NEXUS or PhyloXML. YR-MPE intercepts the export with its own save dialog.
- **Style** — rectangular, transmission or cladogram layout; node sorting (ladderize); tip and internal labels; colour edges or nodes by metadata; collapse zero-length edges; axis (age or time); legend; fonts.
- **Search** — find and highlight clades.
- **Statistics** — lineages-through-time plot, skyline plot and tree statistics.

Right-click a node to re-root. This is the action used by the dating tools' **Choose root**. Rooting in the viewer is display-side unless the exported tree is re-imported into the workspace.

To display branch support, open **Style → Internal node text** and choose `label`. Posterior probabilities, bootstrap percentages and concordance factors appear on the corresponding nodes. For a tree set, double-click the tree-set icon in the workspace; IcyTree opens all member trees and its navigation controls switch between them.

Tip

Export SVG or PNG for figures, and Newick or NEXUS for downstream software. If you reroot a tree for dating, export the rerooted tree and re-import it; otherwise the workspace object keeps its original root.

Very large trees may be slow in the embedded browser. If the viewer shows an error page instead of a tree, the bundled IcyTree assets are missing or the Newick string is malformed; check the Console and the installation.

Chapter 13

Working with AI assistance

YPhylo Workbench exposes the same semantic actions to a human and to an external agent. The agent does not drive the GUI by clicking coordinates or injecting keystrokes. It writes commands to a local file bridge, the workbench executes them on the GUI thread, and the agent reads structured responses and events. This chapter describes the bridge, the action surface and the workflow-recording mechanism.

13.1 What the bridge is

The bridge is a directory containing:

File	Purpose
`commands.jsonl`	Agent-to-workbench requests, one JSON object per line.
`responses.jsonl`	One response per command, correlated by `id`.
`events.jsonl`	Asynchronous progress, log lines, status changes and result-ready events.
`control.json`	Session metadata and the random token; read-only for the agent.
`READY`	Written by the workbench when the session is accepting commands.

The bridge is local-only. It opens no network port, exposes no arbitrary shell execution and requires the random token from `control.json`. The token must not be copied into a world-readable file or committed to a repository. The agent may append to `commands.jsonl`; the workbench polls the file on the Qt main thread, so widget state is never touched from a worker thread.

13.2 Discovering the interface

An agent should inspect before it acts. Five read-only surfaces are available:

- `workbench.menu` — the toolbar tree and a flat list of actions with stable IDs, paths, enabled/checked state and shortcuts.
- `workbench.workspace` — datasets, items, runs, streams and imports, including the provenance links between them.
- `workbench.sidebar` — the Selected basket, recommendations and the current Inspector operations.
- `workbench.skills` — the skill catalog for the installed plugins.
- `workbench.help_doc` — the live document for one plugin, including current values, resolved values and a command preview.

The live help document is the parameter-level contract. A plugin declares `control_bindings()` that maps a GUI widget to the command-line option it represents. For example, the MAFFT direction control maps to `--adjustdirection`, and the RAxML-NG bootstrap control maps to `--bs-trees`. A control with no CLI equivalent is marked GUI-only rather than assigned a fictitious flag.

13.3 Driving a plugin

Once a plugin window is open, the generic action surface is:

- `plugin.inspect_controls` — list controls, current values and CLI mappings;
- `plugin.modify_options` — change one or more options atomically; invalid values are rejected before the plugin runs;
- `plugin.run` — start the analysis;
- `plugin.stop` — request cancellation;
- `plugin.inspect_console` — read the colour-coded console;
- `plugin.export_to_workspace` — materialise the typed result as workspace items.

The agent does not need to know whether a result came from a GUI click or a bridge command: both reach the same handler. Tool-specific APIs, such as the OrthoFinder semantic API, override the generic surface where the tool has a richer contract.

A minimal command sequence for a one-step alignment looks like this:

```
{"id":"open","op":"workbench.invoke_plugin","plugin":"mafft",
 "input":{"item_id":"seq_001"}}
{"id":"set","op":"plugin.modify_options",
 "options":{"mode":"Auto mode (--auto)","direction":"--adjustdirection"}}
{"id":"run","op":"plugin.run"}
{"id":"export","op":"plugin.export_to_workspace"}
```

Each command receives a response with the same `id`; long-running operations also emit `events.jsonl` records. Poll `responses` and `events` until the run reaches a terminal state; do not busy-wait on stdout.

13.4 Recording a workflow

A run log records what happened; a workflow specification records what should happen next. The workbench writes both. A recorded JSONL workflow has an optional header and one object per step:

```
{"schema":"yrmpe.workflow.v1","kind":"workflow",
 "name":"sanger-demo","inputs":{"raw":"raw.fas"}}
{"id":"mafft","op":"mafft.run",
 "outputs":{"alignment":"work/aligned.fas"},
 "params":{"input":"raw.fas","output":"work/aligned.fas",
           "direction":"--adjustdirection"}}
{"id":"tree","op":"iqtree3.run",
 "outputs":{"tree":"work/tree.nwk"},
 "params":{"msa":"work/aligned.fas","model":"MFP",
           "output":"work/tree.nwk"}}
```

The compiler drops read-only discovery actions and failed steps, assigns each output a symbolic placeholder such as `%mafft.alignment.fas%`, rewrites matching inputs and validates the DAG for duplicate producers, unknown placeholders and cycles. The preview renders data items as circles, tools as squares and artifact flow as orthogonal arrows. Clicking a tool node opens its plugin in workflow mode; path inputs are locked to placeholders and the Run button becomes *Close & Apply*.

13.5 Running a workflow

WORKSPACE → Run Workflow... opens the workflow runner. Load a JSONL specification and map each declared input role to a file; the output directory defaults to `workflow/` below the workspace. The runner validates the DAG, the plugin entry for each `op`, the input files and the output directory before starting.

Execution is sequential. For each step the runner creates the ordinary plugin window in headless mode, applies the step parameters through `control_bindings()`, runs the tool, collects the typed result with the plugin's `result_spec()` hook, and rebinds each output role to the file the plugin actually produced. Later placeholders therefore resolve to real files rather than to the paths declared in the JSONL. Progress, command lines and tool output appear in the Console tab; the Procedures tab shows the compiled DAG and Output Preview lists every produced file.

At the end all items are merged into one `ResultSpec` and sent to the workspace, and the runner writes `workflow_run.json` with the plan, per-step status, command line and outputs. The same surface is available to an agent: `workflow.inspect_plan`, `workflow.validate`, `workflow.run`, `workflow.stop`, `workflow.inspect_status` and `workflow.inspect_console`.

13.6 A practical agent recipe

A useful pattern is *inspect, configure, run, verify, record*:

1. Read `workbench.menu` and `workbench.workspace`; confirm the input item and its type.
2. Open the plugin with `workbench.invoke_plugin`; read `plugin.inspect_controls` and the live help document.
3. Change only the options that matter; keep a record of the values.
4. Run, then read `plugin.inspect_console` and the result-ready event. Export the typed result to the workspace.
5. Inspect the new items and provenance; if the result is valid, keep the recorded JSONL as the workflow specification.

This pattern is the basis of the open-task case studies and the plugin authoring/usage benchmark. The agent is expected to recover the reference topology without a prescribed pipeline; the hidden reference trees are never exposed through the bridge.

13.7 Limits and safety

- The bridge is local-only and token-protected. Do not expose it on a network interface.
- There is no arbitrary shell action. A plugin may invoke its own configured binary, but the agent cannot run an unregistered command.
- Long runs are asynchronous. Ask for progress rather than assuming a timeout means failure; read the console and the chain files.
- Human review remains necessary. The agent can validate taxon sets, Newick parsing and file existence, but it cannot judge whether a topology is biologically plausible.
- Do not give an agent a hidden benchmark answer, a reference tree or a world-readable token. Treat those as test-harness data, not context.

Chapter 14

Troubleshooting

Most failures fall into a small number of categories. Start with the Console tab: it shows the exact command, the working directory and the last lines of stdout/stderr.

Symptom	Likely cause	Action
"Executable file not found" or a plugin refuses to start	The binary path in `config.json` is empty, wrong, or points to a moved file.	Run the Initializer's **Auto Config**, or edit the entry in `YR_MPE/config.json`. Verify the path by running the executable with `--version` in a terminal.
Program starts but exits immediately	Wrong binary version or missing runtime library (for example MS-MPI or BEAGLE).	Check the Console for the loader error. Reinstall the bundled runtime; do not mix binaries from different releases.
Model name rejected by RAxML-NG	IQ-TREE syntax differs, or `+FQ` was passed unchanged.	Use a RAxML-NG-compatible model; the plugin normalises `+FQ` to `+FE`, but a hand-edited model string may not be normalised.
ModelTest-NG exits non-zero but writes a report	Known Cygwin exit behaviour after a valid run.	The plugin accepts the report when the `.out` file exists; check the Output Preview rather than the exit code alone.
Partition merge aborts with "Fixing branch lengths not supported"	IQ-TREE 3.0.1 partition-merger limitation.	The plugin retries with `--rcluster 10`. If it still fails, reduce the partition set or run partitions separately.
Cannot select a workspace / Bayesian run refuses to start	Temporary mode is active.	Use **WORKSPACE** → **Select Workspace** → **Select Folder...**. MrBayes and PhyloBayes require a persistent workspace for checkpointing.
Permission denied when writing output	Workspace is on a read-only, network or synchronised folder, or the file is open in another program.	Use a local writable folder; close the file in other applications; avoid OneDrive/Dropbox during long runs.
Long Bayesian run appears frozen	MCMC is running; the window is non-modal and the chain writes to disk periodically.	Check the Console and the `.mcmc` file; do not close the window. Use Stop only if you intend to abandon the run.
MPI fails to start or oversubscribes the CPU	`mpirun` is missing, or the requested processes exceed the machine.	Install the bundled MS-MPI runtime; set Runs × Chains to a number the CPU can supply; reduce chains or runs.
BEAGLE error or very slow likelihoods	Device, precision or scaling is incompatible with the installed GPU/driver.	Switch to CPU, Double precision, Dynamic scaling; update the GPU driver; reduce the thread count.
MUSCLE runs out of memory	Very long sequences.	Use MAFFT, or split the alignment by locus.

Symptom	Likely cause	Action
ASTRAL/wASTRAL aborts on support labels	Composite IQ-TREE labels such as `/1/100` are passed to ASTER's `std::stod`.	The plugin normalises composite labels; if you supply a hand-made tree, use a single numeric support value.
CASTER-site returns an unresolved tree	Too few informative sites.	Use an alignment or supermatrix of at least ∼100 kbp, or choose a gene-tree method.
Negative branch lengths in a DecentTree tree	NJ on a non-additive distance matrix.	Treat as zero for display or apply an explicit zero-clamp; compare with an ML tree before interpreting branch lengths.
Tree imported but not shown in IcyTree	Malformed Newick, missing IcyTree assets, or a rooted/unrooted mismatch.	Validate the Newick with a text editor or Biopython; check the Console; export and re-import the tree.
Output disappeared after closing the workbench	Temporary mode was active.	Use a persistent workspace for anything you want to keep.

14.1 Diagnosing a failed run

1. Read the first Console line. It is the command actually executed.
2. Read the last error lines. A tool may write progress to stderr; the return code, not the stream name, determines failure.
3. Run the same command in a terminal with the same working directory. If it fails there too, the problem is the binary or the input, not the GUI.
4. Check the input format. Most "parse" errors are caused by a FASTA header with spaces, a PHYLIP file with the wrong header count, or a Newick tree broken across lines.
5. Check the taxon labels. Concatenation, ASTRAL and dating all rely on exact label matching.

14.2 Reporting a bug

Include the YR-MPE version (About dialog), operating system, the plugin name, the exact command from the Console, the input file type and size, and the smallest input that reproduces the problem. Do not paste a full multi-gigabyte alignment; a reduced example is more useful.

Chapter 15

Plugin development

15.1 Two kinds of plugin

An **analysis or data plugin** runs a method (an external binary or a built-in Python analysis) and produces one or more workspace objects: alignment, model, distance matrix, tree, tree set, chain, table or dataset. It follows the full contract: a `BasePlugin` subclass, a `BaseProcessThread` subclass when a process is involved, the four inherited tabs, a `ResultSpec` export, and a `PluginEntry.run()` method.

A **viewer or utility plugin** consumes an existing object and does not produce a dataset (IcyTree, MiniTracer, a preview tool). It may use a specialised widget instead of the four-tab layout, but it still declares its metadata, citation keys and entry point, and it never mutates the workspace directly.

15.2 Module and class layout

A typical analysis plugin module is `YR_MPE/plugins/<tool>_plugin.py`. It defines a `<Tool>Thread` (a `BaseProcessThread` subclass), a `<Tool>Plugin` (a `BasePlugin` subclass) and a `<Tool>PluginEntry`. The plugin class implements:

- `init_plugin_info()` — metadata, tool name, citation keys, input and output types;
- `setup_input_tab()` — the parameter form;
- `setup_output_tab()` — the result preview;
- `run_analysis()` — validation, command construction, thread start;
- `stop_analysis()` — cancellation, when the worker is cancellable.

The entry point must accept `import_from=None, import_data=None, workdir=None, **kwargs` and return the plugin widget. Do not rename an existing `source` value or action id without a migration; persisted workspaces and agent scripts depend on them.

15.3 The four tabs

Use the inherited tabs; do not recreate them. Put the parameters in a scrollable **Input & Parameters** form of `QGroupBox` sections, and give **Output Preview** summary counts, a table/tree/text preview, an explicit empty state and an export action. **Console** and **Citation** are inherited: emit the command and output at the correct level, and keep citation keys in the plugin rather than formatted text.

15.4 Metadata, configuration and citations

`init_plugin_info()` must set `plugin_name`, `tool_name`, `citation_keys`, `input_types`, `output_types` and `plugin_path`. A MAFFT plugin, for example, sets `tool_name` to `"MAFFT"`, `citation_keys` to a list containing

`katoh2013mafft`, and `input_types` to a dictionary mapping `"Alignment FASTA"` to the extensions `fas`, `fna`, `fa` and `fasta`. A plugin that wraps an external binary must also have a `config.json` entry with a `name` matching the tool and a `path` relative to `YR_MPE/` (for example, `/bin/mafft/mafft.bat`).

The `name` and `tool_name` must match case-insensitively. Paths are relative to `YR_MPE/` and must not contain a drive letter or home directory. A built-in plugin may override `config()` to return true but must still expose citation keys; a plugin needing a second binary (BLAST+, MPI, BEAGLE) must validate it by name before the run. Citations are data: store one BibTeX/RIS entry per tool with a DOI where available, and keep only the keys in the plugin, in display order.

15.5 Process contract

All external processes must run through `BaseProcessThread` (or an equivalent thread). The thread emits `progress(str)` for user-facing progress, `console_output(...)` for each useful line, `result_ready(object)` for a parsed result, `finished` with the output files exactly once on success, and `error(str)` on failure.

The thread must check `self.is_running` between loop iterations and must never touch a widget or open a dialog. On Windows, use `run_hidden()` or `popen_hidden()` from `YR_MPE/runtime/subprocess_utils.py`; direct `subprocess.run()` or `Popen()` calls allocate a console window under `pythonw`. Pass argument lists rather than shell strings, pass `cwd` explicitly, and classify failures by the return code, including the last stderr lines in the error message.

15.6 Result contract

New analysis plugins must not add a bespoke export signal. Build a `ResultSpec` and call `self.export_result()`. The result contains a stable lowercase `source`, one or more `DatasetSpec` objects with `DatasetItemSpec` items, `artifacts` (output paths), a machine-readable `summary`, non-fatal `warnings` and a `schema` version.

The host owns workspace mutation. The plugin emits a validated result and the workbench converts it into typed objects. Store output paths in the result, not only in the console log.

15.7 Standalone registration

To expose a standalone plugin in the YRTools sidebar, add an entry to `settings.json` with `name`, `placement` (path, priority and favicon), `entry_point` (for example `YR_MPE.icytree:IcyTreePluginEntry`) and `runtime`. The `runtime.type` must be `polypyplug`.

15.8 AI-readiness requirements

A plugin is not finished when it works by mouse. The same analysis must be discoverable and runnable through the semantic surface described in Chapter 13. The following requirements are normative for new analysis plugins; legacy plugins are migrated in the Phase A/B/C order used by the development standard.

15.8.1 Controls must have a machine-readable binding

Every executable-backed analysis plugin must implement `control_bindings()`. Each entry maps one GUI control to the command line it produces:

```
{"id": "direction", "widget": "adjust_direction_combo",
 "label": "Direction adjustment", "kind": "choice",
 "choices": {"No adjustment": [],
             "Adjust direction (--adjustdirection)":
                 ["--adjustdirection"]},
 "cli": ["{value}"],
 "notes": "Nucleotide data only."}
```

The binding is the contract used by `plugin.inspect_controls`, `plugin.modify_options`, the live help document and the skill file. A control with no command-line equivalent must set `"cli": []` and explain why in `notes`; it must not be assigned a fictitious flag. A boolean, integer, choice or path control must declare its type so an agent can validate the value before the run.

15.8.2 Every plugin needs a skill document

Add `YR_MPE/skills/<plugin-id>.md` and register it in the skill manifest. The document should contain, in this order:

1. **Purpose** — what the plugin does and what it does not do;
2. **Inputs** — accepted object types and file formats;
3. **GUI workflow** — the menu path and the minimum steps;
4. **Agent notes** — the semantic actions, the result type and the items written to the workspace;
5. **Interpretation** — what the output means and what it does not mean;
6. **Citation** — the citation keys and any method caveat.

The skill is part of the release, not an afterthought. An agent that can read the menu but not the skill will guess parameter meanings; an agent that can read both can reproduce the GUI workflow.

15.8.3 Results and status must be machine-readable

The result contract in the previous section is also the agent contract. A plugin must return a `ResultSpec`; it must not require an agent to parse the Console or scrape a preview widget. The `source`, item types, summary keys and artifact paths are stable API. Errors must be returned as errors with the return code and the last stderr lines, not as a successful empty result.

For analyses that produce no dataset (for example a terminal summary), the plugin may emit the approved one-line `YMPE_SUMMARY {json}` record (schema `yrmpe.summary.v1`) in addition to the normal display. That record is a fallback for terminal-only tools, not a substitute for `ResultSpec` in an analysis plugin.

15.8.4 Multi-step tools need a semantic API

A simple wrapper can rely on the generic surface (`plugin.inspect_controls`, `modify_options`, `run`, `stop`, `inspect_console`, `export_to_workspace`). A tool with a multi-step workflow (input scan, runtime check, preview, partial results) should expose a tool-specific API in the style of OrthoFinder: `inspect_options`, `modify_options`, `status`, `console`, `preview_results`, `actions` and `agent_invoke`. The tool-specific API overrides the generic surface only where it is richer; the generic actions must still behave sensibly.

15.8.5 Provenance and run ownership

Do not fabricate a `run_id` or a workspace record. The host wrapper creates the run, links inputs and outputs and records the command. The plugin reports the command, working directory, version and artifact paths; the host owns the graph. A plugin that writes outside its work directory, silently overwrites an input, or leaves a temporary file in the workspace root is not AI-ready because the provenance cannot be reconstructed.

15.8.6 AI-assisted development norms

AI tools may draft code, tests, documentation and translations, but a human contributor remains responsible for the commit. In particular:

- **Review the contract, not only the diff.** Check `control_bindings()`, `ResultSpec`, citation keys and error paths against this chapter.

- **Run the tests.** A generated plugin is not accepted until the GUI test, the agent test, the hidden-subprocess test and the Stop test pass on the target Windows build.
- **Do not paste secrets or hidden data.** Tokens, benchmark reference trees and unpublished reviewer material must not be placed in prompts, logs, commit messages or public issues.
- **Do not invent citations or benchmark numbers.** Every citation key must resolve to a real BibTeX/RIS entry; every timing or accuracy claim must come from a recorded run.
- **Disclose the assistance where it matters.** In a pull request or release note, state which parts were AI-assisted and who reviewed them. This is a provenance statement, not a authorship claim.
- **Keep the human decision.** The agent may propose a model, a partition scheme or a topology; the human decides whether the biological result is credible.

15.9 Checklist for a new plugin

1. Does it work standalone and in the workbench, validate before starting a thread, and refuse to close during a run?
2. Does the Console show the command and working directory, and does the preview work without opening an external folder?
3. Does every external process use `run_hidden()` with argument lists and an explicit `cwd`?
4. Does it emit a `ResultSpec`, with output paths in the result?
5. Do the citation keys resolve, and is the tool version recorded?
6. Are failures reported with the last stderr lines, and is the tool listed in its menu and, if user-facing, in this manual?
7. Does `control_bindings()` cover every parameter that maps to the command line, with GUI-only controls marked explicitly?
8. Does the skill document exist, resolve through the skill manifest and describe the result type and the agent actions?
9. Can an agent inspect, modify, run, stop, read the console and export the result without parsing a widget or the Console text?
10. Are the agent bridge token, hidden benchmark data and unpublished material absent from logs, prompts, commits and documentation?

Appendix A

File formats and conventions

Format	Use and conventions
FASTA	Sequences and alignments. The full header after `>` is the taxon identifier for concatenation and matching. Avoid spaces and duplicate headers unless the tool explicitly supports them.
PHYLIP	Distance matrices and alignments. MrBayes and PhyloBayes expect PHYLIP or NEXUS; the workbench converts FASTA automatically where it can.
NEXUS	Partitioned datasets, MrBayes blocks, tree blocks. `charset` definitions use 1-based coordinates.
Newick	Trees. One tree per line for tree sets and ASTRAL input. A Newick string must not contain a line break.
`.mldist`	IQ-TREE lower-triangular ML distance matrix.
`.dist`	Uncorrected p-distance matrix written by the p-distance plugin.
`.p`	MrBayes parameter trace.
`.trace`	PhyloBayes parameter trace.
`.con.tre`	MrBayes consensus tree.
`.treefile`, `.contree`	IQ-TREE and MPBoot best tree and consensus tree.
`.best_scheme.nex`	IQ-TREE partition model scheme.
`.result`	LSD2 dating report.

Throughout, ESS is the effective sample size, PSRF the Gelman–Rubin potential scale reduction factor, and HPD the highest posterior density interval. A “time unit” is whatever unit is used consistently for ages and rates; the software does not convert between years and millions of years. Menu paths use →, and an ellipsis means that a control opens a dialog.

Bibliography

[1] David M. Emms and Steven Kelly. *OrthoFinder: phylogenetic orthology inference for comparative genomics*. Version 2.5.5. 2019. DOI: 10.1186/s13059-019-1816-6.

[2] Kazutaka Katoh and Daron M. Standley. *MAFFT Multiple Sequence Alignment Software Version 7*. Version 7.505. 2013. DOI: 10.1093/molbev/mst010.

[3] Robert C. Edgar. *MUSCLE 5: High-accuracy alignment ensembles enable unbiased assessments of sequence homology and phylogeny*. Version 5.3. 2022. DOI: 10.1038/s41467-022-34630-w.

[4] Fabian Sievers, Andreas Wilm, David Dineen, Toby J. Gibson, Kevin Karplus, Weizhong Li, Rodrigo Lopez, Hamish McWilliam, Michael Remmert, Johannes Söding, Julie D. Thompson, and Desmond G. Higgins. *Fast, scalable generation of high-quality protein multiple sequence alignments using Clustal Omega*. Version 1.2.2. 2011. DOI: 10.1038/msb.2011.75.

[5] Vincent Ranwez, Emmanuel J. P. Douzery, Cédric Cambon, Nathalie Chantret, and Frédéric Delsuc. *MACSE v2: Toolkit for the Alignment of Coding Sequences Accounting for Frameshifts and Stop Codons*. Version 2.07. 2018. DOI: 10.1093/molbev/msy096.

[6] Salvador Capella-Gutiérrez, José M. Silla-Martínez, and Toni Gabaldón. *trimAl: a tool for automated alignment trimming in large-scale phylogenetic analyses*. Version 1.2rev57. 2009. DOI: 10.1093/bioinformatics/btp348.

[7] Gerard Talavera and Jose Castresana. *Improvement of Phylogenies after Removing Divergent and Ambiguously Aligned Blocks from Protein Sequence Alignments*. Version 1.0. 2007. DOI: 10.1080/10635150701472164.

[8] Subha Kalyaanamoorthy, Bui Quang Minh, Thomas K. F. Wong, Arndt von Haeseler, and Lars S. Jermiin. *ModelFinder: fast model selection for accurate phylogenetic estimates*. Version 2.2. 2017. DOI: 10.1038/nmeth.4285.

[9] Thomas K. F. Wong, Nhan Ly-Trong, Huaiyan Ren, Hector Baños, Andrew J. Roger, Edward Susko, Chris Bielow, Nicola De Maio, Nick Goldman, Matthew W. Hahn, Gavin Huttley, Robert Lanfear, and Bui Quang Minh. *IQ-TREE 3: Phylogenomic Inference Software using Complex Evolutionary Models*. Version 3.0.1. 2025. DOI: 10.1093/molbev/msaf117.

[10] Weiwen Wang, James Barbetti, Thomas Wong, Bryan Thornlow, Russ Corbett-Detig, Yatish Turakhia, Robert Lanfear, and Bui Quang Minh. *DecentTree: scalable Neighbour-Joining for the genomic era*. Version 1.0.0. 2023. DOI: 10.1093/bioinformatics/btad536.

[11] Alexey M. Kozlov, Diego Darriba, Tomáš Flouri, Benoit Morel, and Alexandros Stamatakis. *RAxML-NG: a fast, scalable and user-friendly tool for maximum likelihood phylogenetic inference*. Version 1.2.2. 2019. DOI: 10.1093/bioinformatics/btz305.

[12] Fredrik Ronquist, Maxim Teslenko, Paul van der Mark, Daniel L. Ayres, Aaron Darling, Sebastian Höhna, Bret Larget, Liang Liu, Marc A. Suchard, and John P. Huelsenbeck. *MrBayes 3.2: efficient Bayesian phylogenetic inference and model choice across a large model space*. Version 3.2.7a. 2012. DOI: 10.1093/sysbio/sys029.

[13] Nicolas Lartillot, Nicolas Rodrigue, Daniel Stubbs, and Jacques Richer. *PhyloBayes MPI: Phylogenetic Reconstruction with Infinite Mixtures of Profiles in a Parallel Environment*. Version 1.9. 2013. DOI: 10.1093/sysbio/syt030.

[14] Diep Thi Hoang, Le Sy Vinh, Tomáš Flouri, Alexandros Stamatakis, Arndt von Haeseler, and Bui Quang Minh. *MPBoot: fast phylogenetic maximum parsimony tree inference and bootstrap approximation*. Version 1.1.1. 2018. DOI: 10.1186/s12862-018-1131-3.

[15] Chao Zhang, Rasmus Nielsen, and Siavash Mirarab. *ASTER: A Package for Large-Scale Phylogenomic Reconstructions*. Version 1.24.4.8. 2025. DOI: 10.1093/molbev/msaf172.

[16] Chao Zhang, Rasmus Nielsen, and Siavash Mirarab. *CASTER: Direct species tree inference from whole-genome alignments*. Version 1.24.2.6. 2025. DOI: 10.1126/science.adk9688.

[17] Thu-Hien To, Matthieu Jung, Samantha Lycett, and Olivier Gascuel. *LSD2: Fast Dating Using Least-Squares Criteria and Algorithms*. Version 2.4.4. 2016. DOI: 10.1093/sysbio/syv068.

[18] Ziheng Yang. *PAML 4: phylogenetic analysis by maximum likelihood*. Version 4.10.10. 2007. DOI: 10.1093/molbev/msm088.

[19] Ziheng Yang and Bruce Rannala. *MCMCTree*. Version 4.10.10. 2006.

[20] Sergei L. Kosakovsky Pond, Art F. Y. Poon, Ryan Velazquez, Steven Weaver, N. Lance Hepler, Ben Murrell, Stephen D. Shank, Brittany Rife Magalis, Dave Bouvier, Anton Nekrutenko, Sadie Wisotsky, Stephanie J. Spielman, Simon D. W. Frost, and Spencer V. Muse. *HyPhy 2.5—A Customizable Platform for Evolutionary Hypothesis Testing Using Phylogenies*. Version 2.5. 2020. DOI: 10.1093/molbev/msaa122.